```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.CrazyHospital.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S09_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot scene host: register mock services -&gt; run seven steps -&gt; switch to main scene.
    /// On failure it stays on S00_Boot and logs the error (no dead-end while loops).
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.CrazyHospital.Boot
{
    /// &lt;summary&gt;
```

```
/// Seven-step boot sequence: Logger -&gt; Time -&gt; Config -&gt; Save -&gt; Locale -&gt; Ads.Preload -&gt; SwitchM
/// each step logs "[BOOT] n/7 name ok|FAIL". A step exception never blocks later steps.
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &amp;= Run(1, "Logger", LoggerStep);
        ok &amp;= Run(2, "Time", TimeStep);
        ok &amp;= Run(3, "Config", ConfigStep);
        ok &amp;= Run(4, "Save", SaveStep);
        ok &amp;= Run(5, "ILocale", LocaleStep);
        ok &amp;= Run(6, "Ads.Preload", AdsStep);
        ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; // static facade self-initializes on first call
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
    }
    static bool ConfigStep()
    {
```

```
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock = no hot update
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont);
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.CrazyHospital.Boot
{
    /// &lt;summary&gt;
    /// Dev-time mock service set (SY_MOCK scope). All seven boot steps depend on this set.
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; // idempotent: double register would throw
            ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
            ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
            ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
            ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
            ServiceLocator.Register&lt;IAdsService&gt;(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY_MOCK)");
        }
    }
    public sealed class MockTimeService : ITimeService
    {
```

```
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK: device time doubles as server time until the real chain lands.&lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;No cloud: local storage is the authority (zero-server doctrine).&lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// &lt;summary&gt;Missing key falls back to the key itself with a warning.&lt;/summary&gt;
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
        =&gt; new FontSet("G09Main", "G09Main", "G09Main");
}
public sealed class MockAdsService : IAdsService
{
    public event Action&lt;AdLifecycle&gt; OnAdEvent;
    public Task&lt;RewardResult&gt; ShowRewarded(string pointId)
        =&gt; Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
    public void ShowInterstitial(string pointId) { }
    public void ShowBanner(string pointId) { }
    public void Preload(string pointId)
    {
        Logger.Info("ADS", "preload (mock): " + pointId);
```

```
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}
// G09SafeAreaFitter.cs
// Assets/_Game/Scripts/Runtime/Game/G09SafeAreaFitter.cs
// U262 F2 safe-area wiring (BR seat from the C1257 sweep; G03/C1855 +
// G06/C1878 paradigm): notch / rounded-corner / home-indicator insets must
// compact the code-built HUD instead of clipping it. The anchor math lives
// in a pure static so a later CoreCheck section can prove it against
// synthetic rectangles (the editor safeArea is always full-screen - exactly
// the F2 fake-green family root, C1270).
using UnityEngine;
namespace Sy.CrazyHospital
{
    public sealed class SafeAreaFitter : MonoBehaviour
    {
        private Vector2 _lastMin = new Vector2(-1f, -1f);
        private Vector2 _lastSize = Vector2.zero;
        private Vector2 _lastParentSize = Vector2.zero;
        private void OnEnable()
        {
            Apply(Screen.safeArea);
        }
        private void Update()
        {
            Rect sa = Screen.safeArea;
            RectTransform parent = transform.parent as RectTransform;
            Vector2 ps = parent != null ? parent.rect.size : Vector2.zero;
            if (sa.min != _lastMin || sa.size != _lastSize || ps != _lastParentSize)
            {
                Apply(sa);
            }
        }
        private void Apply(Rect sa)
        {
            _lastMin = sa.min;
            _lastSize = sa.size;
            RectTransform parent = transform.parent as RectTransform;
            if (parent != null) _lastParentSize = parent.rect.size;
            ApplyRect((RectTransform)transform, sa);
        }
        // Pure math: map the safe rectangle onto anchor fractions inside a
        // parent of the given size (zero offsets, anchor-only compaction).
        // Returns false when the canvas is not laid out yet so callers skip
        // the fake-green full-screen editor case (C1270 F2 family root).
        // Split out as a pure static so the offline twin probe and a later
        // editor-side CoreCheck section prove the exact same numbers.
        public static bool ComputeAnchors(Vector2 parentSize, Rect sa,
            out Vector2 anchorMin, out Vector2 anchorMax)
```

```
        {
            anchorMin = Vector2.zero;
            anchorMax = Vector2.one;
            if (parentSize.x &lt;= 0f || parentSize.y &lt;= 0f) return false;
            anchorMin = new Vector2(sa.xMin / parentSize.x, sa.yMin / parentSize.y);
            anchorMax = new Vector2(sa.xMax / parentSize.x, sa.yMax / parentSize.y);
            return true;
        }
        public static void ApplyRect(RectTransform target, Rect sa)
        {
            if (target == null) return;
            RectTransform parent = target.parent as RectTransform;
            if (parent == null) return;
            Vector2 aMin, aMax;
            if (!ComputeAnchors(parent.rect.size, sa, out aMin, out aMax)) return;
            target.anchorMin = aMin;
            target.anchorMax = aMax;
            target.offsetMin = Vector2.zero;
            target.offsetMax = Vector2.zero;
        }
    }
}
// HospitalAlbum.cs
// HospitalAlbum.cs - the three-volume album + wonder-drug research
// line (G09-T009, SYS07/SYS08, M07/M08). The album is the collection
// depth face: every absurd disease a patient books, every odd doctor
// hired and every wonder drug researched lands in its frozen volume
// (diseases / doctors / drugs - zero real-medicine vocabulary, every
// row already frozen in HospitalData and HospitalDoctors). The
// collection law is the frozen section-five closed set: a completed
// volume pays income +2% (AlbumBonusPerVolumePct) - the passive face
// the check reconciles; partial volumes only move the rate face. The
// research line (M07) spends gold on the frozen rows and keeps each
// drug's own bonus on the accounting faces (reg_fee_pct /
// bed_fee_pct / cure_rate) - the host folds those into the intake
// economy at the wiring slice, this face keeps the books. Unlocks
// book evt_album_unlock, a purchase books evt_research, a volume
// crossing complete books evt_album_complete once; doctor hires fold
// silently through SyncDoctors (the hire face already booked
// evt_doctor_hire). Second-run inheritance keeps the three volumes
// (InheritAlbumsKept). Pure C#, no MonoBehaviour, zero RNG.
using System;
using System.Collections.Generic;
namespace Sy.CrazyHospital
{
    /// &lt;summary&gt;SYS08: the three-volume album + wonder-drug research
    /// line data face. The host wires RegisterDisease / SyncDoctors /
    /// TryResearch; headless gates drive the static solvers and the
    /// armed faces directly.&lt;/summary&gt;
    public sealed class HospitalAlbum
```

```
{
    // ---- the frozen volume set (diseases / doctors / wonder drugs) ----
    public const string DiseaseVolumeId = "album_disease";
    public const string DrugVolumeId = "album_drug";
    public static string DoctorVolumeId =&gt; HospitalDoctors.DoctorAlbumId;
    readonly HospitalDoctors _doctors;
    readonly List&lt;string&gt; _unlocked = new List&lt;string&gt;();
    readonly HashSet&lt;string&gt; _completed = new HashSet&lt;string&gt;();   // the once-guard of evt_album_comple
    readonly List&lt;string&gt; _funnel = new List&lt;string&gt;();
    int _researchSpendHc;
    public HospitalAlbum(HospitalDoctors doctors = null) { _doctors = doctors; }
    // ---- live faces ----
    public IReadOnlyList&lt;string&gt; Unlocked =&gt; _unlocked;
    public IReadOnlyList&lt;string&gt; Funnel =&gt; _funnel;
    public int ResearchSpendHc =&gt; _researchSpendHc;
    /// &lt;summary&gt;The three volume ids in pack order (diseases /
    /// doctors / wonder drugs).&lt;/summary&gt;
    public static string[] VolumeIds()
        =&gt; new[] { DiseaseVolumeId, HospitalDoctors.DoctorAlbumId, DrugVolumeId };
    /// &lt;summary&gt;Every frozen album entry id of one volume, table
    /// order (the catalog face).&lt;/summary&gt;
    public static List&lt;string&gt; EntriesOf(string volumeId)
    {
        var ids = new List&lt;string&gt;();
        string vol = volumeId ?? "";
        foreach (var d in HospitalData.Diseases()) if (d.albumId == vol) ids.Add(d.id);
        foreach (var d in HospitalDoctors.Doctors()) if (d.albumId == vol) ids.Add(d.id);
        foreach (var d in HospitalData.Research()) if (d.albumId == vol) ids.Add(d.id);
        return ids;
    }
    /// &lt;summary&gt;The volume an entry id belongs to ("" = not a frozen
    /// album entry).&lt;/summary&gt;
    public static string VolumeOf(string entryId)
    {
        string id = entryId ?? "";
        foreach (var d in HospitalData.Diseases()) if (d.id == id) return d.albumId;
        foreach (var d in HospitalDoctors.Doctors()) if (d.id == id) return d.albumId;
        foreach (var d in HospitalData.Research()) if (d.id == id) return d.albumId;
        return "";
    }
    public static int TotalCountOf(string volumeId) =&gt; EntriesOf(volumeId).Count;
    public int UnlockedCountOf(string volumeId)
    {
        string vol = volumeId ?? "";
        int n = 0;
        for (int i = 0; i &lt; _unlocked.Count; i++)
            if (VolumeOf(_unlocked[i]) == vol) n++;
        return n;
    }
    /// &lt;summary&gt;Collection rate of one volume (floor percent).&lt;/summary&gt;
```

```
public int RatePctOf(string volumeId)
    =&gt; SolveRatePct(UnlockedCountOf(volumeId), TotalCountOf(volumeId));
/// &lt;summary&gt;Pure rate solver (headless face: the percent math
/// has no runtime state).&lt;/summary&gt;
public static int SolveRatePct(int unlocked, int total)
    =&gt; total &lt;= 0 ? 0 : unlocked * 100 / total;
public bool IsComplete(string volumeId) =&gt; RatePctOf(volumeId) &gt;= 100;
public int CompletedVolumeCount
{
    get { int n = 0; foreach (var v in VolumeIds()) if (IsComplete(v)) n++; return n; }
}
/// &lt;summary&gt;The section-five passive face: each completed
/// volume pays income +2% (the frozen closed set).&lt;/summary&gt;
public int IncomeBonusPct =&gt; CompletedVolumeCount * HospitalData.AlbumBonusPerVolumePct;
/// &lt;summary&gt;Pure bonus solver (headless face).&lt;/summary&gt;
public static int SolveBonusPct(int completedVolumes)
    =&gt; completedVolumes * HospitalData.AlbumBonusPerVolumePct;
/// &lt;summary&gt;The research accounting face: the sum of bonusValue
/// over the researched rows of one bonus domain
/// (reg_fee_pct / bed_fee_pct / cure_rate) - the wiring slice
/// folds this into the intake economy.&lt;/summary&gt;
public int ResearchBonusOf(string bonusId)
{
    string dom = bonusId ?? "";
    int n = 0;
    foreach (var id in _unlocked)
        foreach (var r in HospitalData.Research())
            if (r.id == id &amp;&amp; r.bonusId == dom) n += r.bonusValue;
    return n;
}
/// &lt;summary&gt;Unlocks one album entry (the comedy-events face:
/// the E06 / E09 album unlocks land here). A new unlock books
/// evt_album_unlock; a volume crossing complete books
/// evt_album_complete once.&lt;/summary&gt;
public bool Unlock(string entryId)
{
    string vol = VolumeOf(entryId);
    if (vol.Length == 0 || _unlocked.Contains(entryId)) return false;
    _unlocked.Add(entryId);
    _funnel.Add("evt_album_unlock@" + vol + ":" + entryId);
    if (IsComplete(vol) &amp;&amp; _completed.Add(vol))
        _funnel.Add("evt_album_complete@" + vol);
    return true;
}
/// &lt;summary&gt;The patient-intake face (M08: the absurd disease
/// registers into the volume the moment its patient books
/// in) - the host calls this from the intake wiring.&lt;/summary&gt;
public bool RegisterDisease(string diseaseId) =&gt; Unlock(diseaseId);
/// &lt;summary&gt;Folds the hired roster into the doctor volume (the
/// hire face already booked evt_doctor_hire, so the sync rides
```

```
/// silent - no duplicate funnel lines).&lt;/summary&gt;
public void SyncDoctors()
{
    if (_doctors == null) return;
    foreach (var d in _doctors.Collection)
    {
        string vol = VolumeOf(d.id);
        if (vol.Length == 0 || _unlocked.Contains(d.id)) continue;
        _unlocked.Add(d.id);
        if (IsComplete(vol) &amp;&amp; _completed.Add(vol))
            _funnel.Add("evt_album_complete@" + vol);
    }
}
/// &lt;summary&gt;The wonder-drug research line (M07): spends gold on
/// one frozen row. Returns the cost booked (0 = refused:
/// unknown row, already researched, or gold short - nothing
/// changes). The purchase books evt_research, the unlock rides
/// Unlock; the caller books the spent gold into its own ledger
/// at the host-wiring slice.&lt;/summary&gt;
public int TryResearch(string drugId, int goldHc)
{
    foreach (var r in HospitalData.Research())
    {
        if (r.id != drugId) continue;
        if (goldHc &lt; r.cost || _unlocked.Contains(drugId)) return 0;
        _researchSpendHc += r.cost;
        _funnel.Add("evt_research@" + drugId + ":" + r.cost);
        Unlock(drugId);
        return r.cost;
    }
    return 0;
}
/// &lt;summary&gt;Second-run inheritance (InheritAlbumsKept):
/// re-seeds the unlocked set silently - a fresh run starts
/// with the volumes it already earned (the flex face).&lt;/summary&gt;
public void InheritFrom(IReadOnlyList&lt;string&gt; entryIds)
{
    if (entryIds == null) return;
    foreach (var id in entryIds)
    {
        string vol = VolumeOf(id);
        if (vol.Length == 0 || _unlocked.Contains(id)) continue;
        _unlocked.Add(id);
        if (IsComplete(vol)) _completed.Add(vol);
    }
}
/// &lt;summary&gt;Gate face: headless checks seed entries without
/// funnel lines (the armed state is the precondition, not the
/// story).&lt;/summary&gt;
public void ArmForGate(params string[] entryIds) =&gt; InheritFrom(entryIds);
```

```
        /// &lt;summary&gt;Canonical fingerprint: the zero-RNG parity
        /// instrument (same script = same bytes).&lt;/summary&gt;
        public string Fingerprint()
        {
            var sb = new System.Text.StringBuilder();
            sb.Append("unlocked=").Append(_unlocked.Count);
            foreach (var v in VolumeIds())
                sb.Append('|').Append(v).Append('=')
                  .Append(UnlockedCountOf(v)).Append('/').Append(TotalCountOf(v));
            sb.Append("|complete=").Append(_completed.Count);
            sb.Append("|income=").Append(IncomeBonusPct);
            sb.Append("|spend=").Append(_researchSpendHc);
            sb.Append("|fun=").Append(_funnel.Count);
            return sb.ToString();
        }
    }
}
// HospitalAlbumTabs.cs
// HospitalAlbumTabs.cs - the three-volume album Tab UI face (G09-T011
// prefix, SYS08): the browsing surface over the frozen volumes - the
// diseases / doctors / wonder-drugs tabs in pack order, one row per
// frozen entry (a collected row carries its comedy title through
// HospitalTexts, a missing row stays behind the ??? mask so the
// collection hunt keeps its punch), the per-volume rate line and the
// closed tab order. The copy rides existing keys only: the diseases
// tab header = the album title, the doctors tab = the duty roster,
// the drugs tab = the wonder-drug lab, and every row title routes
// through the row id as its own key - zero new keys, the zh parity
// law stays green. Pure C#, zero RNG, zero scene state, no event on
// browsing: the future UGUI host renders TabSpec, the headless gate
// drives the static solvers.
using System.Collections.Generic;
namespace Sy.CrazyHospital
{
    /// &lt;summary&gt;SYS08: the album Tab UI composition face. The host
    /// renders BuildAll / BuildTab; headless checks drive the pure
    /// solvers (SolveHeader / SolveTab / VolumeAt).&lt;/summary&gt;
    public sealed class HospitalAlbumTabs
    {
        public const int TabCount = 3;              // diseases / doctors / drugs
        public const string LockedMask = "???";    // the silhouette glyph - a mask, not a sentence
        /// &lt;summary&gt;One album row: the frozen entry id, its comedy
        /// title (masked while missing) and the collected state.&lt;/summary&gt;
        public struct RowSpec
        {
            public string EntryId;
            public string Title;
            public bool Unlocked;
        }
        /// &lt;summary&gt;One tab: the volume id, its header caption, the
```

```
/// rate line faces and the row list in table order.&lt;/summary&gt;
public struct TabSpec
{
    public string VolumeId;
    public string Header;
    public int RatePct;
    public int Unlocked;
    public int Total;
    public RowSpec[] Rows;
}
/// &lt;summary&gt;Tab order law (closed set): the pack order of the
/// three volumes, the index clamped on both ends.&lt;/summary&gt;
public static string VolumeAt(int tabIndex)
{
    var ids = HospitalAlbum.VolumeIds();
    int i = tabIndex &lt; 0 ? 0 : (tabIndex &gt;= ids.Length ? ids.Length - 1 : tabIndex);
    return ids[i];
}
/// &lt;summary&gt;The tab index of one volume (-1 = not a frozen
/// volume).&lt;/summary&gt;
public static int TabIndexOf(string volumeId)
{
    var ids = HospitalAlbum.VolumeIds();
    for (int i = 0; i &lt; ids.Length; i++) if (ids[i] == volumeId) return i;
    return -1;
}
/// &lt;summary&gt;Header solver (closed set): each volume's caption
/// rides an existing copy key - no new key, the zh parity law
/// stays green.&lt;/summary&gt;
public static string SolveHeader(string volumeId)
{
    if (volumeId == HospitalAlbum.DiseaseVolumeId)
        return HospitalTexts.EnText(HospitalTexts.KeyAlbumTitle);
    if (volumeId == HospitalAlbum.DoctorVolumeId)
        return HospitalTexts.EnText(HospitalTexts.KeyDutyRoster);
    if (volumeId == HospitalAlbum.DrugVolumeId)
        return HospitalTexts.EnText(HospitalTexts.KeyResearchTitle);
    return volumeId ?? "";
}
/// &lt;summary&gt;Pure tab solver (the headless face): one row per
/// frozen entry in table order - collected rows carry their
/// comedy title, missing rows stay behind the mask; the rate
/// faces count only this volume's entries.&lt;/summary&gt;
public static TabSpec SolveTab(string volumeId, IReadOnlyList&lt;string&gt; unlocked)
{
    var entries = HospitalAlbum.EntriesOf(volumeId);
    var set = unlocked != null ? new HashSet&lt;string&gt;(unlocked) : null;
    var rows = new RowSpec[entries.Count];
    int n = 0;
    for (int i = 0; i &lt; entries.Count; i++)
```

```
            {
                bool has = set != null &amp;amp;&amp;amp; set.Contains(entries[i]);
                if (has) n++;
                rows[i] = new RowSpec
                {
                    EntryId = entries[i],
                    Title = has ? HospitalTexts.EnText(entries[i]) : LockedMask,
                    Unlocked = has
                };
            }
            return new TabSpec
            {
                VolumeId = volumeId,
                Header = SolveHeader(volumeId),
                RatePct = HospitalAlbum.SolveRatePct(n, entries.Count),
                Unlocked = n,
                Total = entries.Count,
                Rows = rows
            };
        }
        /// &lt;summary&gt;Live fold face: reads the wired album's unlocked
        /// set (a null album folds to the honest empty-collection
        /// face - every row masked, zero rate).&lt;/summary&gt;
        public static TabSpec BuildTab(HospitalAlbum album, string volumeId)
            =&gt; SolveTab(volumeId, album != null ? album.Unlocked : null);
        /// &lt;summary&gt;All three tabs in pack order (the render face).&lt;/summary&gt;
        public static TabSpec[] BuildAll(HospitalAlbum album)
        {
            var tabs = new TabSpec[TabCount];
            for (int i = 0; i &lt; TabCount; i++) tabs[i] = BuildTab(album, VolumeAt(i));
            return tabs;
        }
    }
}
// HospitalArtView.cs
// HospitalArtView.cs - S09_Main art host face (G09 S2.5-T020 slice b2).
// G18 ViewHost paradigm (HallViewHost): self-installs on the main scene,
// builds its display canvas from code and consumes the C1078 ArtCatalog
// stems art-first (X027 iron rule: real art wins, null falls back to the
// procedural block - no face ever depends on art presence). Slice scope =
// the consumption-face proof: facade backdrop mapped by hospital rank,
// director hero slot, the seven ready portrait cards (the wangdali card
// lands with the A11R swap-in), the event-card frame slot (its keyed-bg
// residue shows until the A14R swap-in - art-first swaps with zero code
// change) and the ui-kit skeleton strip. Live stage binding (rank + cast
// from the running HospitalStage) lands with the stage-host slice; the
// test hooks below are the one true op routers until then.
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
```

```
using UnityEngine.UI;
using Logger = Sy.Core.Logger;
namespace Sy.CrazyHospital
{
    public sealed class HospitalArtView : MonoBehaviour
    {
        public const string SceneName = "S09_Main";
        // art contract keys = ArtCatalog keys = art book sec7 (C1078)
        public const string DirectorFemaleStem = "g09_director_f";
        public const string DirectorMaleStem = "g09_director_m";
        public const string EventFrameStem = "g09_eventcard_frame";
        public const string UiKitStem = "g09_ui_kit";
        /// &lt;summary&gt;The seven portrait cards: 4 clean cuts + 3 with keyed-bg
        /// edge residue (C1079 probe) that stay art-first until their re-roll
        /// swap-in; the wangdali card lands with the A11R swap-in.&lt;/summary&gt;
        public static readonly string[] PortraitStems =
        {
            DirectorFemaleStem, DirectorMaleStem,
            "g09_doc_beauty", "g09_doc_moyu", "g09_doc_threesec",
            "g09_pat_hiccup", "g09_pat_restless",
        };
        const float RefW = 1080f, RefH = 1920f;
        // fallback tints ride the HospitalPlaceholder palette family (mint
        // accent brand face) - only visible while a stem misses
        static readonly Color TintDirector = new Color(0.43f, 0.85f, 0.78f);
        static readonly Color TintDoctor = new Color(0.80f, 0.88f, 0.95f);
        static readonly Color TintPatient = new Color(0.92f, 0.80f, 0.88f);
        static readonly Color TintEvent = new Color(0.95f, 0.90f, 0.66f);
        static readonly Color TintKit = new Color(0.14f, 0.18f, 0.20f, 0.92f);
        // ---- pure mapping faces (EditMode-testable, presenter never invents) ----
        /// &lt;summary&gt;Facade art key of a hospital rank (1-5; out-of-range folds to R1).&lt;/summary&gt;
        public static string FacadeStemOf(int rank)
        {
            int i = rank - 1;
            if (i &lt; 0 || i &gt;= 5) i = 0;
            return "g09_facade_r" + (i + 1);
        }
        /// &lt;summary&gt;Director hero art key by gender face.&lt;/summary&gt;
        public static string DirectorStemOf(bool female)
        {
            return female ? DirectorFemaleStem : DirectorMaleStem;
        }
        // ---- runtime face ----
        int _rank = 1;
        bool _directorFemale = true;
        RectTransform _canvasRect;
        Image _facade;
        Image _director;
        Image _eventFrame;
        Image _uiKit;
```

```
Image[] _portraitCards;
[RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.BeforeSceneLoad)]
static void AutoInstall()
{
    SceneManager.sceneLoaded -= OnSceneLoaded;
    SceneManager.sceneLoaded += OnSceneLoaded;
}
static void OnSceneLoaded(Scene scene, LoadSceneMode mode)
{
    if (scene.name != SceneName) return;
    if (FindObjectOfType&lt;HospitalArtView&gt;() != null) return;
    var go = new GameObject("HospitalArtView");
    go.AddComponent&lt;HospitalArtView&gt;();
}
void Awake()
{
    BuildUi();
    RefreshFaces();
    Logger.Info("G09.View", "art view installed (S2.5-T020 slice b2 consumption face online)");
}
// ---- test hooks = the one true op routers (stage-host slice lands next) ----
public void TestSetRank(int rank) { _rank = rank; RefreshFaces(); }
public void TestSetDirector(bool female) { _directorFemale = female; RefreshFaces(); }
public string FacadeKey { get { return FacadeStemOf(_rank); } }
public string DirectorKey { get { return DirectorStemOf(_directorFemale); } }
public bool FacadeArtPresent { get { return SlotIsArt(_facade, FacadeKey); } }
public bool DirectorArtPresent { get { return SlotIsArt(_director, DirectorKey); } }
public bool EventFrameArtPresent { get { return SlotIsArt(_eventFrame, EventFrameStem); } }
public bool UiKitArtPresent { get { return SlotIsArt(_uiKit, UiKitStem); } }
/// &lt;summary&gt;How many of the seven portrait cards carry real art.&lt;/summary&gt;
public int PortraitArtCount()
{
    int n = 0;
    if (_portraitCards == null) return 0;
    for (int i = 0; i &lt; _portraitCards.Length; i++)
        if (SlotIsArt(_portraitCards[i], PortraitStems[i])) n++;
    return n;
}
static bool SlotIsArt(Image slot, string stem)
{
    return slot != null &amp;&amp; slot.sprite != null &amp;&amp; slot.sprite.name == stem;
}
void RefreshFaces()
{
    if (_facade != null)
        _facade.sprite = HospitalPlaceholder.Facade(_rank);
    if (_director != null)
        _director.sprite = HospitalPlaceholder.Face(DirectorKey, TintDirector, 64);
    if (_eventFrame != null)
        _eventFrame.sprite = HospitalPlaceholder.Face(EventFrameStem, TintEvent, 48);
```

```
    if (_uiKit != null)
        _uiKit.sprite = HospitalPlaceholder.Face(UiKitStem, TintKit, 64);
    if (_portraitCards != null)
    {
        for (int i = 0; i &lt; _portraitCards.Length; i++)
        {
            bool isPatient = PortraitStems[i].StartsWith("g09_pat_");
            _portraitCards[i].sprite = HospitalPlaceholder.Face(
                PortraitStems[i], isPatient ? TintPatient : TintDoctor, 32);
        }
    }
}
// ---------------- UGUI construction (display-only canvas: no raycaster,
// every image raycastTarget=false - it never swallows scene input) ----------------
void BuildUi()
{
    var root = new GameObject("HospitalArtCanvas", typeof(Canvas));
    var canvas = root.GetComponent&lt;Canvas&gt;();
    canvas.renderMode = RenderMode.ScreenSpaceOverlay;
    canvas.sortingOrder = 10;   // above the scene's own canvas; display-only
    var scaler = root.AddComponent&lt;CanvasScaler&gt;();
    scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
    scaler.referenceResolution = new Vector2(RefW, RefH);
    scaler.matchWidthOrHeight = 0.5f;
    _canvasRect = (RectTransform)root.transform;
    // U262 F2 safe-area wiring (BR seat from the C1257 sweep; G03/C1855
    // + G06/C1878 paradigm): the whole display tree parents onto a
    // fitter-driven safe root so notch / rounded-corner /
    // home-indicator insets compact the code-built slots instead of
    // clipping them. Retargeting the shared _canvasRect field means
    // zero Slot() call-site changes.
    var safeGo = new GameObject("G09SafeRoot", typeof(RectTransform));
    var safeRect = (RectTransform)safeGo.transform;
    safeRect.SetParent(root.transform, false);
    safeRect.anchorMin = Vector2.zero;
    safeRect.anchorMax = Vector2.one;
    safeRect.offsetMin = Vector2.zero;
    safeRect.offsetMax = Vector2.zero;
    safeGo.AddComponent&lt;SafeAreaFitter&gt;();
    _canvasRect = safeRect;
    // facade backdrop (rank-mapped vertical face; fill-slot mode with
    // preserveAspect - the 540x960 source is the same 9:16 as the ref)
    _facade = Slot("facade", new Vector2(RefW * 0.5f, RefH * 0.5f), new Vector2(RefW, RefH));
    _facade.transform.SetAsFirstSibling();
    // director hero slot (top-left)
    _director = Slot("director_hero", new Vector2(220f, 1560f), new Vector2(300f, 300f));
    // event-card frame slot (top-right; keyed-bg residue shows until A14R)
    _eventFrame = Slot("event_frame", new Vector2(860f, 1560f), new Vector2(360f, 280f));
    // seven ready portrait cards (one row; fill-slot, aspect 1:1 slots)
    _portraitCards = new Image[PortraitStems.Length];
```

```
            for (int i = 0; i &lt; PortraitStems.Length; i++)
                _portraitCards[i] = Slot("card_" + PortraitStems[i],
                    new Vector2(120f + i * 140f, 1120f), new Vector2(128f, 128f));
            // ui-kit skeleton strip (bottom band)
            _uiKit = Slot("ui_kit", new Vector2(RefW * 0.5f, 190f), new Vector2(1000f, 320f));
        }
        Image Slot(string name, Vector2 canvasPos, Vector2 size)
        {
            var go = new GameObject(name, typeof(RectTransform), typeof(Image));
            var rect = (RectTransform)go.transform;
            rect.SetParent(_canvasRect, false);
            rect.sizeDelta = size;
            rect.anchoredPosition = ToLocal(canvasPos);
            var img = go.GetComponent&lt;Image&gt;();
            img.raycastTarget = false;
            img.preserveAspect = true;   // fill-slot mode chosen explicitly (asset-config skill sec5)
            img.color = Color.white;
            return img;
        }
        static Vector2 ToLocal(Vector2 canvasPos)
        {
            return canvasPos - new Vector2(RefW, RefH) * 0.5f;
        }
    }
}
// HospitalAudio.cs
// HospitalAudio.cs - the T021 audio face (the G10 TradeAudio paradigm
// applied to the clinic): the stamp-to-stem map, the per-project
// AudioCatalog (the X027 art-catalog paradigm applied to audio), and a
// null-safe playback service. The catalog asset is built by the editor
// tool (AudioSetup) from Assets/_Game/Audio; runtime consumers MUST
// treat a missing catalog/clip as "skip silently" so headless checks
// stay green on any project state (G02Sfx headless law).
//
// Stem semantics come from the G09 audio manifest (S1A1/A2/A3 batches):
// three loop channels (main BGM baked with the signature ECG-beep +
// discharge-whistle hook, lobby bed, night-ward bed) and twelve
// one-shots. The stage stamps and the engine funnels ride composed ids
// ("evt_promote@r2:evt_review_level1") - StemForStamp resolves the base
// id before the first '@' so every composed seat lands on its stem.
using System;
using System.Collections.Generic;
using UnityEngine;
namespace Sy.CrazyHospital
{
    /// &lt;summary&gt;
    /// Generated-audio catalog (T021, X027 paradigm): the AudioSetup
    /// editor tool scans Assets/_Game/Audio and serializes clip
    /// references here so runtime code resolves audio by filename stem.
    /// The asset lives at Assets/_Game/Resources/AudioCatalog.asset
```

```
/// (Resources keeps runtime lookup build-safe with zero
/// Addressables wiring for playtest slices).
/// &lt;/summary&gt;
[CreateAssetMenu(fileName = "AudioCatalog", menuName = "Sy/CrazyHospital Audio Catalog")]
public sealed class AudioCatalog : ScriptableObject
{
    [Serializable]
    public sealed class Entry
    {
        public string key;      // filename stem, e.g. "g09_sfx_coin"
        public AudioClip clip;  // direct reference keeps builds self-contained
    }
    public List&lt;Entry&gt; entries = new List&lt;Entry&gt;();
    Dictionary&lt;string, AudioClip&gt; _index;
    void OnEnable()
    {
        // ScriptableObject instances can be revived with a stale cache;
        // drop it so the next Get() rebuilds from entries.
        _index = null;
    }
    public AudioClip Get(string key)
    {
        if (string.IsNullOrEmpty(key)) return null;
        if (_index == null)
        {
            _index = new Dictionary&lt;string, AudioClip&gt;(entries.Count, StringComparer.Ordinal);
            for (int i = 0; i &lt; entries.Count; i++)
            {
                var e = entries[i];
                if (e != null &amp;&amp; !string.IsNullOrEmpty(e.key) &amp;&amp; e.clip != null &amp;&amp; !_index.C
                    _index[e.key] = e.clip;
            }
        }
        return _index.TryGetValue(key, out var c) ? c : null;
    }
}
/// &lt;summary&gt;
/// Runtime lookup helper. Every consumer MUST treat null as "clip
/// missing -&gt; skip" so headless checks stay green on any project
/// state (catalog absent, audio pruned, name typos).
/// &lt;/summary&gt;
public static class AudioRuntime
{
    static AudioCatalog _catalog;
    public static AudioCatalog Catalog
    {
        get
        {
            if (_catalog == null) _catalog = Resources.Load&lt;AudioCatalog&gt;("AudioCatalog");
            return _catalog;
```

```
        }
    }
    public static AudioClip Get(string key)
    {
        var c = Catalog;
        return c != null ? c.Get(key) : null;
    }
}
/// &lt;summary&gt;
/// The audio face of the clinic (T021): maps the opening-script
/// stamps, the rank-review funnel lines, and the named face cues to
/// catalog stems, and plays them null-safe on hidden hosts (never
/// throws headless). The signature hook (ECG triple beep + the
/// four-note discharge whistle, the earworm red line) is baked into
/// bgm_main; the discharge whistle also owns its own one-shot leg
/// (sfx_launch, the catapult-discharge comedy cue).
/// &lt;/summary&gt;
public static class HospitalAudio
{
    // ---- delivered stems (the closed set, 15) ----
    public const string StemBgmMain = "g09_bgm_main";       // clinic-day loop BGM (hook baked in)
    public const string StemAmbLobby = "g09_amb_lobby";     // registration lobby bed
    public const string StemAmbWard = "g09_amb_ward";      // night ward bed (offline summary face)
    public const string StemSfxRegister = "g09_sfx_register"; // the registration stamp thunk
    public const string StemSfxCoin = "g09_sfx_coin";       // the coin family (registration + bed income)
    public const string StemSfxUpgrade = "g09_sfx_upgrade"; // the department upgrade blast
    public const string StemSfxEvent = "g09_sfx_event";     // the comedy event-card pop
    public const string StemSfxInspect = "g09_sfx_inspect"; // the surprise-inspection whistle
    public const string StemSfxStamp = "g09_sfx_stamp";     // the promotion seal ceremony
    public const string StemSfxLaunch = "g09_sfx_launch";  // the catapult discharge whoosh
    public const string StemSfxBone = "g09_sfx_bone";       // the orthopedics puzzle clicks
    public const string StemSfxBanner = "g09_sfx_banner";   // the pennant banner wall mount
    public const string StemSfxEmergency = "g09_sfx_emergency"; // the night emergency-tide alarm
    public const string StemSfxAlbum = "g09_sfx_album";     // the album unlock page ding
    public const string StemSfxRank = "g09_sfx_rank";       // the review-verdict gong
    /// &lt;summary&gt;The signature coin-chime stamp fired by the opening script.&lt;/summary&gt;
    public const string CoinSfxStamp = "coin_sfx";
    /// &lt;summary&gt;The hook stamp owns t0 and starts the loop channels.&lt;/summary&gt;
    public static readonly string StampHospitalHook = HospitalEventIds.HospitalHook;
    /// &lt;summary&gt;Stage stamps / engine funnel bases -&gt; one-shot stems.
    /// The composed funnel seats ("evt_promote@r2:...", "evt_band_change@rep_banner")
    /// resolve through the base-id law in StemForStamp.&lt;/summary&gt;
    static readonly Dictionary&lt;string, string&gt; StampMap = new Dictionary&lt;string, string&gt;
    {
        { HospitalEventIds.FirstRegistration, StemSfxRegister },  // the queue head registers
        { CoinSfxStamp, StemSfxCoin },                             // the coin popup face
        { HospitalEventIds.DeptOpened, StemSfxUpgrade },           // the first department opens
        { HospitalEventIds.EventCardOffer, StemSfxEvent },         // the comedy card offer
        { HospitalEventIds.EventCard, StemSfxEvent },             // the comedy card resolve
        { HospitalEventIds.Promote, StemSfxStamp },                // the promotion seal ceremony
```

```
    { HospitalEventIds.Inspect, StemSfxInspect },              // the rectification inspection face
    { HospitalEventIds.BandChange, StemSfxBanner },            // the banner wall grows at the top band
    { HospitalEventIds.AlbumUnlock, StemSfxAlbum },            // the encyclopedia entry registers
    { HospitalEventIds.AlbumComplete, StemSfxAlbum },          // the volume completion face
    { HospitalEventIds.OfflineSettle, StemSfxEmergency },      // the offline night tide settles
    { HospitalEventIds.FacadeLit, StemSfxStamp },             // the R5 top-grade stamp (ending face)
};
/// &lt;summary&gt;Named face cues (non-stamp faces) -&gt; one-shot stems.&lt;/summary&gt;
public const string CueReviewVerdict = "review_verdict";   // the review settle gong (pass or fail)
public const string CueDischarge = "discharge_launch";      // the bed turn completes: the patient flies home
public const string CueBonePuzzle = "bone_puzzle";          // the orthopedics mini-game beat
static readonly Dictionary&lt;string, string&gt; CueMap = new Dictionary&lt;string, string&gt;
{
    { CueReviewVerdict, StemSfxRank },
    { CueDischarge, StemSfxLaunch },
    { CueBonePuzzle, StemSfxBone },
};
public static IReadOnlyDictionary&lt;string, string&gt; Stamps =&gt; StampMap;
public static IReadOnlyDictionary&lt;string, string&gt; Cues =&gt; CueMap;
/// &lt;summary&gt;The loop-channel stems (main BGM + lobby bed + night ward bed).&lt;/summary&gt;
public static readonly string[] Channels = { StemBgmMain, StemAmbLobby, StemAmbWard };
/// &lt;summary&gt;Resolves a stamp or composed funnel seat to its stem.
/// The base-id law: the token before the first '@' is the
/// registry base ("evt_promote@r2:evt_review_level1" -&gt; "evt_promote").&lt;/summary&gt;
public static string StemForStamp(string stampId)
{
    if (string.IsNullOrEmpty(stampId)) return null;
    string baseId = stampId;
    int at = stampId.IndexOf('@');
    if (at &gt; 0) baseId = stampId.Substring(0, at);
    return baseId != null &amp;&amp; StampMap.TryGetValue(baseId, out var s) ? s : null;
}
public static string StemForCue(string cueId)
    =&gt; cueId != null &amp;&amp; CueMap.TryGetValue(cueId, out var s) ? s : null;
// ---- playback (null-safe, headless-safe) ----
static AudioSource _host;
static readonly Dictionary&lt;string, AudioSource&gt; _loopChannels = new Dictionary&lt;string, AudioSource&gt;(
static AudioSource Host()
{
    if (_host == null)
    {
        var go = new GameObject("G09AudioHost");
        UnityEngine.Object.DontDestroyOnLoad(go);
        _host = go.AddComponent&lt;AudioSource&gt;();
    }
    return _host;
}
/// &lt;summary&gt;Fire-and-forget one-shot; false = missing/skipped, never throws.&lt;/summary&gt;
public static bool Play(string stem)
{
```

```
            try
            {
                var clip = AudioRuntime.Get(stem);
                if (clip == null) return false;
                Host().PlayOneShot(clip);
                return true;
            }
            catch (Exception)
            {
                return false;   // headless/batchmode: silence is the legal fallback
            }
        }
        public static bool PlayStamp(string stampId) =&gt; Play(StemForStamp(stampId));
        public static bool PlayCue(string cueId) =&gt; Play(StemForCue(cueId));
        /// &lt;summary&gt;Start (or keep) a loop channel; false = missing/skipped.&lt;/summary&gt;
        public static bool StartChannel(string stem)
        {
            try
            {
                var clip = AudioRuntime.Get(stem);
                if (clip == null) return false;
                if (!_loopChannels.TryGetValue(stem, out var src) || src == null)
                {
                    var go = new GameObject("G09Audio_" + stem);
                    UnityEngine.Object.DontDestroyOnLoad(go);
                    src = go.AddComponent&lt;AudioSource&gt;();
                    _loopChannels[stem] = src;
                }
                src.clip = clip;
                src.loop = true;
                if (!src.isPlaying) src.Play();
                return true;
            }
            catch (Exception)
            {
                return false;
            }
        }
        /// &lt;summary&gt;The main-face entry: the hook stamp owns t0, the
        /// clinic BGM and the lobby bed ride in with it (the channel law).&lt;/summary&gt;
        public static bool StartMainFace()
        {
            bool bgm = StartChannel(StemBgmMain);
            bool amb = StartChannel(StemAmbLobby);
            return bgm || amb;
        }
        /// &lt;summary&gt;The night-face entry: the offline summary and the
        /// night-shift faces ride the ward bed (amb_ward).&lt;/summary&gt;
        public static bool StartNightFace() =&gt; StartChannel(StemAmbWard);
    }
```

```
}
// HospitalData.cs
// HospitalData.cs - frozen data schemas and tables for Crazy Hospital (G09-T004).
// The numeric pack doc (G09 v1, the S0 doc face) is the source of truth and
// this file is its compiled mirror: schema freeze means the structural laws
// live here as code and DataSchemaCheck proves every law on every row.
// Values are sweep-tunable later (bot sweeps re-freeze revisions through the
// doc pack), the structure only moves with a schema version bump. Zero
// server faces: the tables ship in the build (red line 12). Compliance
// faces ride the freeze: the whole disease set stays fictional, the
// catapult-discharge law keeps the death face age-friendly, and the green
// cross family avoids any real medical emblem.
namespace Sy.CrazyHospital
{
    /// &lt;summary&gt;One hospital rank row: the intake economy face (M01) and the
    /// department slot face (M02) frozen on one row per rank R1-R5.&lt;/summary&gt;
    public sealed class RankRow
    {
        public int rank;           // 1-5 (R1 unlicensed clinic .. R5 top-grade stamped)
        public int baseTraffic;    // patients per game-minute at the par reputation band
        public int regFeeHc;       // registration fee in gold hundredths per patient
        public int bedFeeHc;       // bed fee in gold hundredths per second per bed
        public int incomeCapHc;    // full-rank income ceiling in gold hundredths per second
        public int slots;          // preset department slots
        public int firstSlotCost;  // gold cost of the first slot
        public int deptOutputHc;   // department output in gold hundredths per second per slot
    }
    /// &lt;summary&gt;One rank-review gate (M04, dual track): the hard metrics AND
    /// the review event together - a promotion is a new gameplay ticket,
    /// never a wall. Failing a review only arms the rectification comedy and
    /// the next review window.&lt;/summary&gt;
    public sealed class ReviewGateDef
    {
        public int fromRank;
        public int toRank;
        public int minDepts;
        public int minCureRatePct;
        public int minReputation;
        public string reviewEventId = "";
    }
    /// &lt;summary&gt;One reputation band row (M06, the second numeric line,
    /// 0-100 segments with a visible comedy face per band).&lt;/summary&gt;
    public sealed class ReputationBand
    {
        public int low;         // inclusive
        public int high;        // inclusive
        public string id = "";
    }
    /// &lt;summary&gt;One wonder-drug research row (M07): absurd fiction only.&lt;/summary&gt;
    public sealed class ResearchDef
```

```
{
    public string id = "";
    public int cost;        // research cost in gold
    public string bonusId = "";  // reg_fee_pct / bed_fee_pct / cure_rate
    public int bonusValue;  // bonus magnitude (percent points, or points for the cure rate)
    public string albumId = "";
}
/// &lt;summary&gt;One fictional disease album row (M08 - zero real-medicine
/// vocabulary anywhere in the set).&lt;/summary&gt;
public sealed class DiseaseDef
{
    public string id = "";
    public string albumId = "";
}
public static class HospitalData
{
    public const int SchemaVersion = 1;
    public const int RankCount = 5;
    public const int GateCount = 4;      // R1-&gt;R2 .. R4-&gt;R5
    public const int BandCount = 4;     // the reputation 0-100 partition
    public const int ResearchCount = 3;
    public const int DiseaseCount = 5;
    public const int ReputationMin = 0;
    public const int ReputationMax = 100;
    public const int AlbumVolumeCount = 3;   // diseases / doctors / wonder drugs
    // ---- section 1: the intake economy - traffic is a reputation
    // function. actual = base * (0.4 + reputation/100 * 1.2), frozen in
    // basis points: factorBp = 4000 + rep * 120. Reputation 50 = the par
    // band at exactly 1.0; high reputation lines the patients up, low
    // reputation drains the street (a two-way visible face). ----
    public const int TrafficFactorBaseBp = 4000;
    public const int TrafficFactorRepBp = 120;
    public static int TrafficAt(int baseTraffic, int reputation)
    {
        int r = ClampReputation(reputation);
        return baseTraffic * (TrafficFactorBaseBp + r * TrafficFactorRepBp) / 10000;
    }
    static int ClampReputation(int v)
        =&gt; v &lt; ReputationMin ? ReputationMin : (v &gt; ReputationMax ? ReputationMax : v);
    // ---- pacing anchors (the red line 11 assertion sources) ----
    public const int FirstCoinMaxMs = 1500;    // first registration coin &lt;= 1.5s
    public const int P1FirstDeptSeconds = 60;   // first department open + first patient booked
    public const int P50QueueSeconds = 300;      // three departments + the comedy queue
    public const int AdsPerDayMin = 3;          // north-star ad expectation
    // ---- section 2: the department curve laws ----
    public const bool PresetSlotsOnly = true;    // pick WHAT opens, never WHERE (no coordinates in the schema)
    public const int SlotCostLinearLaw = 1;     // n-th slot cost = firstSlotCost * n (linear, never exponential)
    public const int UpgradeOutputPercentPerLevel = 150;  // output *1.5 per level
    public const int UpgradeCostPercentPerLevel = 200;    // cost *2 per level
    public const int UpgradeMaxLevel = 3;                // the upgrade ladder caps at 3
```

```
public static int SlotCost(RankRow row, int slotIndex)
    =&gt; row.firstSlotCost * slotIndex * SlotCostLinearLaw;   // 1-based slot index
// ---- section 3: the cure-rate law (age-friendly death translation)
public const bool CatapultDischargeCountsCured = true; // the catapult ride home counts as a cure
// ---- section 4: the reputation modifier closed set (v1) ----
public const int ReviewRepSwing = 3;      // review settlement swings +/-3
public const int ComedyRepSwingMin = 1;    // comedy events swing +/-1..2 (E06-E10 proposal band)
public const int ComedyRepSwingMax = 2;
public const bool RepAutoDecay = false;    // v1: no automatic decay (bot sweeps may revisit)
// ---- section 5: the album collection bonus ----
public const int AlbumBonusPerVolumePct = 2;   // each completed volume = income +2% (v1 proposal)
// ---- section 6: offline (SYS09, the night emergency tide) ----
public const int OfflineRatePercent = 50;          // offline advances at half pace (series law)
public const int NightEmergencyFactorBp = 15000;  // offline-window traffic *1.5 (night = emergency peak)
public const int OfflineCapHours = 12;            // per-absence cap
public const string OfflineDoubleCategory = "double"; // AdCap family category
public const int DoubleOfferPerSpan = 1;          // one double offer per absence span
public const int DoubleWinMinPer200 = 1;          // frequency window 1-5 per 200 (G08 T009 paradigm)
public const int DoubleWinMaxPer200 = 5;
// ---- section 7: second-run inheritance (M10, the top-grade ending) ----
public const int InheritGoldPercent = 20;        // floor
public const bool InheritDoctorCards = true;    // the expert team follows the new dean
public const bool InheritAlbumsKept = true;     // the three albums persist (the flex face)
public const bool InheritRankReset = true;      // the spine restarts at the unlicensed clinic
/// &lt;summary&gt;Rank names (English telemetry face; the zh copy rides the texts table).&lt;/summary&gt;
public static readonly string[] RankNames =
    { "UNLICENSED CLINIC", "LEVEL-1 HOSPITAL", "LEVEL-2 HOSPITAL", "LEVEL-3 HOSPITAL", "TOP-GRADE STAMPED" };
// ---- the rank table: intake economy + slot curve on one row ----
static readonly RankRow[] RankTable =
{
    new RankRow { rank = 1, baseTraffic =  6, regFeeHc = 200, bedFeeHc =  30, incomeCapHc =  120, slots =  1, fi
    new RankRow { rank = 2, baseTraffic = 12, regFeeHc = 200, bedFeeHc =  50, incomeCapHc =  300, slots =  3, fi
    new RankRow { rank = 3, baseTraffic = 20, regFeeHc = 300, bedFeeHc =  80, incomeCapHc =  700, slots =  5, fi
    new RankRow { rank = 4, baseTraffic = 30, regFeeHc = 300, bedFeeHc = 120, incomeCapHc = 1400, slots =  8, fi
    new RankRow { rank = 5, baseTraffic = 45, regFeeHc = 400, bedFeeHc = 160, incomeCapHc = 2500, slots = 12, fi
};
public static RankRow[] Ranks() =&gt; (RankRow[])RankTable.Clone();
// ---- rank-review gates (dual track; the review names ride the
// texts table, the events arm the review comedy) ----
static readonly ReviewGateDef[] GateTable =
{
    new ReviewGateDef { fromRank = 1, toRank = 2, minDepts = 1, minCureRatePct = 60, minReputation = 30, reviewE
    new ReviewGateDef { fromRank = 2, toRank = 3, minDepts = 3, minCureRatePct = 65, minReputation = 45, reviewE
    new ReviewGateDef { fromRank = 3, toRank = 4, minDepts = 5, minCureRatePct = 70, minReputation = 55, reviewE
    new ReviewGateDef { fromRank = 4, toRank = 5, minDepts = 8, minCureRatePct = 75, minReputation = 65, reviewE
};
public static ReviewGateDef[] Gates() =&gt; (ReviewGateDef[])GateTable.Clone();
// ---- reputation bands (the 0-100 partition, four visible faces) ----
static readonly ReputationBand[] BandTable =
{
```

```
    new ReputationBand { low = 0,  high = 24,  id = "rep_bad" },    // review storm + bad-review comedy cards
    new ReputationBand { low = 25, high = 49,  id = "rep_plain" },  // quiet streets, business as usual
    new ReputationBand { low = 50, high = 74,  id = "rep_good" },    // steady patients
    new ReputationBand { low = 75, high = 100, id = "rep_banner" }, // banner-wall queues
};
public static ReputationBand[] Bands() =&gt; (ReputationBand[])BandTable.Clone();
public static ReputationBand BandFor(int reputation)
{
    foreach (var b in BandTable)
        if (reputation &gt;= b.low &amp;&amp; reputation &lt;= b.high) return b;
    return BandTable[0];
}
// ---- wonder-drug research rows (M07, absurd fiction only) ----
static readonly ResearchDef[] ResearchTable =
{
    new ResearchDef { id = "drug_hiccup_cola",     cost = 100, bonusId = "reg_fee_pct", bonusValue = 1, albumId
    new ResearchDef { id = "drug_sleepwalk_patch", cost = 250, bonusId = "bed_fee_pct", bonusValue = 1, albumId
    new ResearchDef { id = "drug_shock_sugar",     cost = 500, bonusId = "cure_rate",  bonusValue = 1, albumId =
};
public static ResearchDef[] Research() =&gt; (ResearchDef[])ResearchTable.Clone();
// ---- the fictional disease album (M08, zero real-medicine vocabulary) ----
static readonly DiseaseDef[] DiseaseTable =
{
    new DiseaseDef { id = "dis_hiccup_bug",      albumId = "album_disease" },  // the hiccup bug
    new DiseaseDef { id = "dis_gummy_bone",     albumId = "album_disease" },  // gummy-bone syndrome
    new DiseaseDef { id = "dis_monday",         albumId = "album_disease" },  // Monday syndrome
    new DiseaseDef { id = "dis_head_grass",      albumId = "album_disease" },  // grass on the head (coder speci
    new DiseaseDef { id = "dis_cactus_accident", albumId = "album_disease" },  // the cactus-diet accident
};
public static DiseaseDef[] Diseases() =&gt; (DiseaseDef[])DiseaseTable.Clone();
/// &lt;summary&gt;Canonical signature - the dual-derive byte-parity
/// instrument (the schema freeze proof): two derives of the frozen
/// tables agree here or nowhere.&lt;/summary&gt;
public static string Signature()
{
    var sb = new System.Text.StringBuilder();
    sb.Append("v").Append(SchemaVersion);
    sb.Append("|ranks:");
    foreach (var r in RankTable)
        sb.Append(r.rank).Append('/')
          .Append(r.baseTraffic).Append(',').Append(r.regFeeHc).Append(',').Append(r.bedFeeHc).Append(',')
          .Append(r.incomeCapHc).Append(',').Append(r.slots).Append(',').Append(r.firstSlotCost).Append(',')
          .Append(r.deptOutputHc).Append(';');
    sb.Append("|traffic:").Append(TrafficFactorBaseBp).Append('+').Append(TrafficFactorRepBp);
    sb.Append("|pacing:").Append(FirstCoinMaxMs).Append(',').Append(P1FirstDeptSeconds).Append(',').Append(P50Qu
    sb.Append("|slots:").Append(PresetSlotsOnly).Append(',').Append(SlotCostLinearLaw).Append(',')
      .Append(UpgradeOutputPercentPerLevel).Append(',').Append(UpgradeCostPercentPerLevel).Append(',').Append(Up
    sb.Append("|cure:").Append(CatapultDischargeCountsCured);
    sb.Append("|rep:").Append(ReviewRepSwing).Append(',').Append(ComedyRepSwingMin).Append(',')
      .Append(ComedyRepSwingMax).Append(',').Append(RepAutoDecay);
```

```
            foreach (var b in BandTable) sb.Append(',').Append(b.id).Append(':').Append(b.low).Append('-').Append(b.high
            sb.Append("|album:").Append(AlbumVolumeCount).Append(',').Append(AlbumBonusPerVolumePct);
            sb.Append("|offline:").Append(OfflineRatePercent).Append(',').Append(NightEmergencyFactorBp).Append(',')
              .Append(OfflineCapHours).Append(',').Append(OfflineDoubleCategory).Append(',')
              .Append(DoubleOfferPerSpan).Append(',').Append(DoubleWinMinPer200).Append(',').Append(DoubleWinMaxPer200);
            sb.Append("|inherit:").Append(InheritGoldPercent).Append(',').Append(InheritDoctorCards).Append(',')
              .Append(InheritAlbumsKept).Append(',').Append(InheritRankReset);
            sb.Append("|gates:");
            foreach (var g in GateTable)
                sb.Append(g.fromRank).Append('&gt;').Append(g.toRank).Append('/')
                  .Append(g.minDepts).Append(',').Append(g.minCureRatePct).Append(',').Append(g.minReputation)
                  .Append(',').Append(g.reviewEventId).Append(';');
            sb.Append("|research:");
            foreach (var d in ResearchTable)
                sb.Append(d.id).Append('/').Append(d.cost).Append('/').Append(d.bonusId).Append('+').Append(d.bonusValue
            sb.Append("|diseases:");
            foreach (var d in DiseaseTable) sb.Append(d.id).Append('@').Append(d.albumId).Append(';');
            return sb.ToString();
        }
    }
}
// HospitalDoctors.cs
// HospitalDoctors.cs - the quack-doctor hiring engine (G09-T006, SYS03).
// The desk consumes the frozen doctor table (numeric pack section ten, the
// T006 freeze): every card carries one passive on a named clinic domain
// plus one pure-comedy side effect - the mischief is a laugh beat, never a
// management burden (TH simplification law). The ten-card hiring spree is
// ad-funded (zero IAP, zero real gacha, GachaEnabled stays false) and rides
// the G08 T009 three-piece law on the draw category: HasCategoryBudget
// precheck and ShowRewarded consumption share one policy instance and one
// day bucket, the evt_ad funnel books offer -&gt; show -&gt; complete -&gt; landed
// in order, and a capped tap refuses BEFORE the ad ever plays. The duty
// roster is automated: seats auto-fill in hire order and the roster cap
// reads the open department slots (on-duty count never exceeds open
// departments; a benched card never pays its passive). Draw order is a
// deterministic rotation (zero RNG); a duplicate card books the
// "already on payroll" comedy count. Pure C#, no MonoBehaviour.
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Platform;
namespace Sy.CrazyHospital
{
    /// &lt;summary&gt;One quack-doctor card (the frozen section-ten table row).&lt;/summary&gt;
    public sealed class DoctorDef
    {
        public string id = "";
        public string passiveId = "";   // the clinic-domain bonus id
        public string quirkId = "";     // the comedy beat key (zero numeric weight)
        public string albumId = "";    // the doctor album volume
```

```
}
/// &lt;summary&gt;The on-duty clinic profile every passive folds into (identity
/// defaults keep every T005 frozen number untouched - an empty desk pays
/// exactly the pre-T006 economy).&lt;/summary&gt;
public sealed class DoctorProfile
{
    public int TurnBp = 10000;       // three-second doc: the bed turn x0.75 (12s -&gt; 9s)
    public int RegSurchargeHc;        // quack remedy: +20 hc registration surcharge
    public int DeptBp = 10000;        // slacking director: department output x1.15
    public int BedBp = 10000;         // wang-dali: the bed fee x1.2
    public int TrafficBp = 10000;     // night nurse: street traffic +10% (round half up)
    public int CapBp = 10000;         // beauty chief: the income ceiling x1.1
    public bool IsIdentity
        =&gt; TurnBp == 10000 &amp;&amp; RegSurchargeHc == 0 &amp;&amp; DeptBp == 10000
           &amp;&amp; BedBp == 10000 &amp;&amp; TrafficBp == 10000 &amp;&amp; CapBp == 10000;
}
/// &lt;summary&gt;SYS03: the hiring desk. The scene host wires the ten-draw
/// CTA to TryDrawTen, the roster to the open-slot count via SyncRoster,
/// and the desk to the clinic via ApplyTo; headless gates drive all
/// three directly.&lt;/summary&gt;
public sealed class HospitalDoctors
{
    /// &lt;summary&gt;Draw point id (T006): draws from the g09_card_draw
    /// 3/day category (AdCategories.CategoryOfPoint authority, g09_
    /// prefix keeps id namespaces isolated).&lt;/summary&gt;
    public const string DrawPoint = "g09_card_draw";
    public const int CardsPerDraw = 10;
    /// &lt;summary&gt;The frozen compliance face: zero IAP, zero real gacha -
    /// the ten-card spree is ad-funded comedy, never a paid pull.&lt;/summary&gt;
    public const bool GachaEnabled = false;
    public const string DoctorAlbumId = "album_doctor";
    readonly AdCapPolicy _adCap;
    readonly IAdsService _ads;
    readonly Func&lt;string&gt; _dayKey;
    readonly List&lt;DoctorDef&gt; _collected = new List&lt;DoctorDef&gt;();
    readonly List&lt;DoctorDef&gt; _roster = new List&lt;DoctorDef&gt;();
    readonly HashSet&lt;string&gt; _quirkFired = new HashSet&lt;string&gt;();
    int _pullCount;
    int _dupeTotal;
    public HospitalDoctors(AdCapPolicy adCap, IAdsService ads, Func&lt;string&gt; dayKey = null)
    {
        _adCap = adCap;
        _ads = ads;
        _dayKey = dayKey ?? AdCapPolicy.TodayKey;
        if (ads != null) ads.OnAdEvent += OnAdLifecycle;
    }
    // ---- live faces ----
    public IReadOnlyList&lt;DoctorDef&gt; Collection =&gt; _collected;
    public IReadOnlyList&lt;DoctorDef&gt; Roster =&gt; _roster;
    public int DupeTotal =&gt; _dupeTotal;
```

```
public int PullCount =&gt; _pullCount;
public int CollectedCount =&gt; _collected.Count;
/// &lt;summary&gt;Live draw-category precheck: always-fresh budget face on
/// the same policy instance and day bucket the registered service
/// consumes from, so precheck and consume can never disagree.&lt;/summary&gt;
public bool OfferBudgetOk
    =&gt; _adCap != null &amp;&amp; _adCap.HasCategoryBudget(_dayKey(), AdCategories.CategoryOfPoint(DrawPoint))
/// &lt;summary&gt;The ten-card hiring spree CTA: budget precheck - rewarded
/// view - the deterministic ten-card deal. The ad only ever plays
/// with budget left; a capped tap refuses BEFORE the ad (no arming,
/// no show, no budget). An incomplete view changes nothing.&lt;/summary&gt;
public async Task&lt;bool&gt; TryDrawTen()
{
    if (!OfferBudgetOk)
    {
        Funnel.Add("evt_ad_offer_capped@p" + _pullCount);
        return false;
    }
    Funnel.Add("evt_ad_offer@p" + _pullCount);
    RewardResult r = await _ads.ShowRewarded(DrawPoint);
    if (r == null || !r.Completed) return false;   // evt_ad_fail rides the lifecycle face; retry stays open
    int deal = _pullCount;
    _pullCount++;
    var table = Doctors();
    for (int j = 0; j &lt; CardsPerDraw; j++)
    {
        var card = table[(deal * CardsPerDraw + j) % table.Length];
        if (IndexOf(_collected, card) &lt; 0)
        {
            _collected.Add(card);
            Funnel.Add("evt_doctor_hire@p" + deal + ":" + card.id);
        }
        else
        {
            _dupeTotal++;
            Funnel.Add("evt_doctor_dupe@p" + deal + ":" + card.id);
        }
    }
    Funnel.Add("evt_ad_landed@p" + deal);
    return true;
}
/// &lt;summary&gt;Duty-roster automation: seats take the collected doctors
/// in hire order (first hired, first on duty), capped by the OPEN
/// department slots - on-duty count never exceeds open departments
/// and a benched card never pays its passive. The quirk beat fires
/// exactly once per doctor on the first day on duty.&lt;/summary&gt;
public void SyncRoster(int openSlots)
{
    _roster.Clear();
    int cap = openSlots &lt; 0 ? 0 : openSlots;
```

```
    foreach (var d in _collected)
    {
        if (_roster.Count &gt;= cap) break;
        _roster.Add(d);
        if (_quirkFired.Add(d.id))
            Funnel.Add("evt_doctor_quirk@p" + _pullCount + ":" + d.id);
    }
}
/// &lt;summary&gt;Second-run inheritance (numeric pack section eight: the
/// expert team follows the new dean): kept doctor cards land on the
/// fresh desk's collection directly - no draw, no ad burn.
/// Idempotent per id; the roster sync stays the host's call.&lt;/summary&gt;
public void InheritFrom(IReadOnlyList&lt;string&gt; doctorIds)
{
    if (doctorIds == null) return;
    var table = Doctors();
    foreach (var id in doctorIds)
        foreach (var d in table)
            if (d.id == id &amp;&amp; IndexOf(_collected, d) &lt; 0)
            {
                _collected.Add(d);
                Funnel.Add("evt_doctor_inherit@" + d.id);
            }
}
/// &lt;summary&gt;The on-duty clinic profile folded from the roster
/// (collection without a duty seat never pays).&lt;/summary&gt;
public DoctorProfile ProfileOf()
{
    var p = new DoctorProfile();
    foreach (var d in _roster)
    {
        switch (d.passiveId)
        {
            case "passive_turn_x075": p.TurnBp = 7500; break;      // three-second doc: "NEXT!" - 12s turn -&gt;
            case "passive_reg_plus20": p.RegSurchargeHc = 20; break; // quack remedy: the ancestral-remedy surch
            case "passive_dept_x115": p.DeptBp = 11500; break;    // slacking director: the dept runs smoother w
            case "passive_bed_x120": p.BedBp = 12000; break;       // wang-dali: the bone-bolting craft premium
            case "passive_traffic_x110": p.TrafficBp = 11000; break; // night nurse: never sleepy on the night s
            case "passive_cap_x110": p.CapBp = 11000; break;      // beauty chief: the high-ticket makeover orde
        }
    }
    return p;
}
/// &lt;summary&gt;Wires the clinic: the on-duty profile folds into the
/// intake engine's doctor face (an empty duty roster keeps the
/// identity economy byte-identical).&lt;/summary&gt;
public void ApplyTo(HospitalIntake intake)
{
    if (intake == null) return;
    var p = ProfileOf();
```

```
    var d = intake.Doctors;
    d.TurnBp = p.TurnBp;
    d.RegSurchargeHc = p.RegSurchargeHc;
    d.DeptBp = p.DeptBp;
    d.BedBp = p.BedBp;
    d.TrafficBp = p.TrafficBp;
    d.CapBp = p.CapBp;
}
/// &lt;summary&gt;evt_ad funnel (G08 T009 paradigm): lifecycle events of
/// the draw point land in the funnel list for gate assertions.&lt;/summary&gt;
void OnAdLifecycle(AdLifecycle e)
{
    if (e.PointId != DrawPoint) return;
    Funnel.Add("evt_ad_" + e.Type.ToString().ToLowerInvariant() + "@p" + _pullCount);
}
public readonly List&lt;string&gt; Funnel = new List&lt;string&gt;();
// ---- the frozen section-ten doctor table (T006 freeze, mirror of
// the numeric pack doc; order IS the seating order) ----
static readonly DoctorDef[] DoctorTable =
{
    new DoctorDef { id = "doc_threesec",     passiveId = "passive_turn_x075",     quirkId = "quirk_threesec",
    new DoctorDef { id = "doc_remedy_quack", passiveId = "passive_reg_plus20",    quirkId = "quirk_remedy_quack"
    new DoctorDef { id = "doc_moyu",         passiveId = "passive_dept_x115",     quirkId = "quirk_moyu",
    new DoctorDef { id = "doc_wangdali",    passiveId = "passive_bed_x120",      quirkId = "quirk_wangdali",
    new DoctorDef { id = "doc_night_nurse", passiveId = "passive_traffic_x110",  quirkId = "quirk_night_nurse",
    new DoctorDef { id = "doc_beauty_chief",passiveId = "passive_cap_x110",       quirkId = "quirk_beauty_chief"
};
public static DoctorDef[] Doctors() =&gt; (DoctorDef[])DoctorTable.Clone();
public static DoctorDef DoctorOf(string id)
{
    foreach (var d in DoctorTable) if (d.id == id) return d;
    return null;
}
static int IndexOf(List&lt;DoctorDef&gt; list, DoctorDef d)
{
    for (int i = 0; i &lt; list.Count; i++) if (ReferenceEquals(list[i], d)) return i;
    return -1;
}
/// &lt;summary&gt;Canonical fingerprint: the zero-RNG parity instrument.&lt;/summary&gt;
public string Fingerprint()
{
    var sb = new System.Text.StringBuilder();
    sb.Append("pulls=").Append(_pullCount);
    sb.Append("|col=");
    foreach (var d in _collected) sb.Append(d.id).Append(',');
    sb.Append("|duty=");
    foreach (var d in _roster) sb.Append(d.id).Append(',');
    sb.Append("|dupes=").Append(_dupeTotal);
    sb.Append("|fun=").Append(Funnel.Count);
    return sb.ToString();
```

```
        }
    }
}
// HospitalEnding.cs
// HospitalEnding.cs - the R5 top-grade-stamp ending + the NG+ carry
// (G09-T011 closing sub-slice, SYS08 / M10, numeric pack section
// seven). The ending face is the facade fully-lit ceremony: the
// vertical facade IS the growth progress bar (S09_Main, one lit floor
// per rank state, R1 clinic -&gt; R5 top-grade tower), and the R5 stamp
// crossing lights the last floor - the once-guard books evt_facade_lit
// and arms the face the render layer consumes at the S2.5 wiring. The
// NG+ carry is the frozen section-seven closed set: gold carries 20%
// floored, the expert team follows the new dean (every collected
// doctor card kept - InheritFrom books evt_doctor_inherit on the fresh
// desk), the three album volumes ride InheritAlbumsKept (the cross-run
// brag face, silent re-plant), and the rank / event flags reset by
// simply NOT being carried - a fresh run starts at the R1 identity.
// The ceremony copy rides the pre-frozen keys only (grade title /
// carry label / new-journey pair), zero new text keys. Pure C#, no
// MonoBehaviour, zero RNG.
using System;
using System.Collections.Generic;
namespace Sy.CrazyHospital
{
    /// &lt;summary&gt;The NG+ carry pack (frozen section seven): the three
    /// carried faces and nothing else - gold 20% floored, the kept
    /// doctor cards, the kept album rows. Rank and event flags are
    /// the NOT-carried face by construction.&lt;/summary&gt;
    public struct NgPlusCarry
    {
        public int GoldCarryHc;
        public List&lt;string&gt; AlbumIds;
        public List&lt;string&gt; DoctorIds;
    }
    /// &lt;summary&gt;SYS08 / M10: the facade-lit ending engine + the NG+
    /// carry coordinator. The host wires Observe(rank) into the
    /// promotion settle and SolveCarry / ApplyCarry into the
    /// new-journey gate; headless gates drive the static solvers and
    /// the armed faces directly.&lt;/summary&gt;
    public sealed class HospitalEnding
    {
        // ---- the frozen ending laws (closed set, zero new numbers) ----
        public const int EndingRank = 5;          // R5 = the top-grade stamp (the ladder cap)
        public const int FacadeFloors = 5;        // the vertical facade states R1-R5
        public const int GoldCarryPercent = 20;    // section seven: gold carries 20% floored
        public const string FacadeLitEvent = "evt_facade_lit";
        public const string NgPlusCarryEvent = "evt_ngplus_carry";
        readonly List&lt;string&gt; _funnel = new List&lt;string&gt;();
        int _litFloors = 1;                        // the facade growth bar (max rank observed)
        bool _facadeLit;                           // the once-guard of evt_facade_lit
```

```
            sb.Append("|slots=").Append(_openSlots).Append('/').Append(_rank.slots);
            sb.Append("|choice=").Append(_lastChoice);
            sb.Append("|queue=").Append(_queue);
            sb.Append("|arrived=").Append(_arrivalTotal);
            sb.Append("|patients=").Append(_patientTotal);
            sb.Append("|cured=").Append(_curedTotal);
            sb.Append("|beds=");
            for (int i = 0; i &lt; _beds.Length; i++) sb.Append(_beds[i].ToString("0.###")).Append(',');
            sb.Append("|led=").Append(_ledger.regHc).Append(',').Append(_ledger.bedHc)
              .Append(',').Append(_ledger.deptHc).Append(',').Append(_ledger.spilledHc);
            sb.Append("|maxwin=").Append(_maxWindowAcc.ToString("0.###"));
            return sb.ToString();
        }
    }
}
// HospitalOffline.cs
// HospitalOffline.cs - the offline settler (G09-T010, SYS09 M09): the
// "while you were gone" face. One settle = one absence span: the span caps
// at the frozen 12h, folds to the 50% pace (the series law), then the night
// emergency tide lands its extra walk-ins up front - offline-window
// traffic rides at x1.5 (the night = emergency peak narrative, the
// night-shift nurse's street bonus reads through the very traffic the
// tide folds) - and the whole hospital walks the live sim clock the
// online game runs on: department output, bed fees, the treatment
// countdowns and the event cards all keep rolling without you, the night
// nurse runs the desk (a freed bed takes the next sleeping companion),
// a pending event card simply waits on the dean's desk. The hall-slept-
// full face is the summary law: the tide count, the pace, the queue
// left outside and the gold walk all land on the while-you-were-gone
// card. The double offer rides the shared double 5/day category (one
// offer per span, the frozen 1-5/200 window band): the armed guard
// first, the HasCategoryBudget precheck second (one policy instance,
// one day bucket - precheck and consume can never disagree), the
// rewarded view third, and only the completed view books the earned
// gold a second time through the event ledger line - the wallet law
// (gold == start - slot costs + booked ledger lines) holds by
// construction. A capped tap refuses BEFORE the ad ever plays; an
// aborted view changes nothing and the retry stays open. Pure C#
// zero-RNG: the same absence span always settles byte-identical.
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Text;
using System.Threading.Tasks;
using Sy.Platform;
namespace Sy.CrazyHospital
{
    public sealed class HospitalOffline
    {
        /// &lt;summary&gt;The offline double point id (T010): rides the shared
```

```
/// double category through AdCategories.CategoryOfPoint authority
/// (the g09_ prefix keeps id namespaces isolated, the
/// g08/g10/g11/g13/g14 doubler precedents).&lt;/summary&gt;
public const string DoublePoint = "g09_offline_double";
/// &lt;summary&gt;The night-desk walk slice: the nurse checks the hall
/// once a second (a freed bed takes the next companion within the
/// same second it freed in).&lt;/summary&gt;
public const double WalkSliceSeconds = 1.0;
readonly HospitalStage _stage;
readonly AdCapPolicy _adCap;
readonly IAdsService _ads;
readonly Func&lt;string&gt; _dayKey;
readonly List&lt;string&gt; _funnel = new List&lt;string&gt;();
readonly List&lt;string&gt; _summary = new List&lt;string&gt;();
int _spanIndex;
double _spanSeconds;
double _simSeconds;
int _tideArrivals;
int _eventsRolledBefore;
int _goldBeforeHc;
int _earnHc;
bool _armed;
bool _claimed;
public HospitalOffline(HospitalStage stage, AdCapPolicy adCap, IAdsService ads, Func&lt;string&gt; dayKey = null
{
    _stage = stage;
    _adCap = adCap;
    _ads = ads;
    _dayKey = dayKey ?? AdCapPolicy.TodayKey;
    if (ads != null) ads.OnAdEvent += OnAdLifecycle;
}
// ---- live faces ----
public HospitalStage Stage =&gt; _stage;
public double SpanSeconds =&gt; _spanSeconds;
public double SimSeconds =&gt; _simSeconds;
public int TideArrivals =&gt; _tideArrivals;
public int EarnHc =&gt; _earnHc;
public bool OfferArmed =&gt; _armed;
public bool Claimed =&gt; _claimed;
public IReadOnlyList&lt;string&gt; Funnel =&gt; _funnel;
public IReadOnlyList&lt;string&gt; SummaryLines =&gt; _summary;
string SpanTag =&gt; "s" + _spanIndex;
/// &lt;summary&gt;The night-tide fold: the extra walk-ins the x1.5
/// offline-window traffic books over the natural street stream
/// (the natural stream keeps rolling inside the walk itself).
/// Deterministic off the settle-start traffic snapshot.&lt;/summary&gt;
public static int TideArrivalsFor(int trafficPerMinute, double simSeconds)
    =&gt; (int)Math.Ceiling(trafficPerMinute * simSeconds / 60.0
        * (HospitalData.NightEmergencyFactorBp - 10000) / 10000.0);
/// &lt;summary&gt;Settles one absence span: caps at the frozen 12h,
```

```
    /// folds to the 50% pace, front-loads the night tide queue, walks
    /// the whole stage through the sim seconds with the night nurse on
    /// the desk, then writes the while-you-were-gone summary and arms
    /// the double offer once (only when the span earned something to
    /// double). A zero absence books nothing at all: head line only,
    /// no funnel, the offer never arms (the zero-span law).&lt;/summary&gt;
    public void Settle(double absentSeconds)
    {
        _spanIndex++;
        _claimed = false;
        _armed = false;
        _earnHc = 0;
        _tideArrivals = 0;
        _summary.Clear();
        _summary.Add(HospitalTexts.EnText(HospitalTexts.KeyOfflineSummary));
        if (absentSeconds &lt;= 0)
        {
            // the zero-span law: zero sim, zero funnel, head line only
            _spanSeconds = 0;
            _simSeconds = 0;
            return;
        }
        _spanSeconds = Math.Min(absentSeconds, HospitalData.OfflineCapHours * 3600.0);
        _simSeconds = _spanSeconds * HospitalData.OfflineRatePercent / 100.0;
        var intake = _stage.Intake;
        _tideArrivals = TideArrivalsFor(intake.TrafficPerMinute, _simSeconds);
        _eventsRolledBefore = _stage.Events.RollCount;
        _goldBeforeHc = intake.GoldHc;
        intake.AddStreetQueue(_tideArrivals);   // the tide front-load: they lined up while you slept
        _funnel.Add("evt_offline_settle@" + SpanTag + ":"
            + _spanSeconds.ToString("0.#", CultureInfo.InvariantCulture) + "&gt;"
            + _simSeconds.ToString("0.#", CultureInfo.InvariantCulture)
            + ":tide" + _tideArrivals);
        double remaining = _simSeconds;
        while (remaining &gt; 0.0000001)
        {
            double step = Math.Min(WalkSliceSeconds, remaining);
            _stage.Advance(step);
            remaining -= step;
            // the night desk: a freed bed takes the next companion
            if (intake.OpenSlots &gt; 0 &amp;&amp; intake.QueueCount &gt; 0)
                intake.TryRegisterQueueHead();
        }
        _earnHc = intake.GoldHc - _goldBeforeHc;
        _armed = _earnHc &gt; 0;   // one offer per span, only when there is something to double
        BuildSummary();
    }
    void BuildSummary()
    {
        var intake = _stage.Intake;
```

```
    _summary.Add(HospitalTexts.EnText(HospitalTexts.KeyNightTide) + " " + _tideArrivals
        + " EXTRA WALK-INS - THE HALL SLEPT FULL OF COMPANIONS");
    _summary.Add("away " + HoursStr(_spanSeconds) + "h -&gt; " + HoursStr(_simSeconds) + "h at "
        + HospitalData.OfflineRatePercent + "% pace");
    int rolled = _stage.Events.RollCount - _eventsRolledBefore;
    if (rolled &gt; 0 || _stage.Events.PendingCard != null)
        _summary.Add("the night desk kept busy: " + rolled + " event cards rolled, "
            + (_stage.Events.PendingCard != null ? "one is waiting on your desk" : "none left waiting"));
    _summary.Add("the street never emptied: " + intake.QueueCount + " still lining up outside");
    _summary.Add("gold " + HcStr(_goldBeforeHc) + " -&gt; " + HcStr(intake.GoldHc)
        + " (" + (_earnHc &gt;= 0 ? "+" : "") + HcStr(_earnHc) + ")");
}
/// &lt;summary&gt;The double offer claim (the shared double category,
/// one offer per span): the armed guard first (a dead tap changes
/// nothing - zero state, zero budget), the category precheck
/// second, the rewarded view third; only the completed view books
/// the earned gold a second time through the event ledger line.
/// A capped tap refuses BEFORE the ad; an aborted view keeps the
/// span armed for the retry.&lt;/summary&gt;
public async Task&lt;bool&gt; ClaimDouble()
{
    if (!_armed) return false;   // the dead tap: zero state, zero budget
    if (_adCap == null || _ads == null) return false;
    if (!_adCap.HasCategoryBudget(_dayKey(), AdCategories.CategoryOfPoint(DoublePoint)))
    {
        _funnel.Add("evt_ad_offer_capped@" + SpanTag);
        return false;   // the capped tap refuses BEFORE the ad ever plays
    }
    _funnel.Add("evt_ad_offer@" + SpanTag);
    RewardResult r = await _ads.ShowRewarded(DoublePoint);
    if (r == null || !r.Completed) return false;   // the aborted view: armed stays, the retry window stays open
    _stage.Intake.BookEventGoldHc(_earnHc);   // the x2 injection rides the event ledger line - the wallet law h
    _funnel.Add("evt_offline_double@" + SpanTag + ":" + _earnHc);
    _armed = false;
    _claimed = true;
    return true;
}
/// &lt;summary&gt;evt_ad funnel (the registered-service lifecycle face):
/// show/complete/fail of the double point land in order.&lt;/summary&gt;
void OnAdLifecycle(AdLifecycle e)
{
    if (e.PointId != DoublePoint) return;
    _funnel.Add("evt_ad_" + e.Type.ToString().ToLowerInvariant() + "@" + SpanTag);
}
static string HoursStr(double seconds)
    =&gt; (seconds / 3600.0).ToString("0.##", CultureInfo.InvariantCulture);
static string HcStr(long hc)
    =&gt; (hc / 100.0).ToString("0.##", CultureInfo.InvariantCulture);
/// &lt;summary&gt;Canonical fingerprint - the zero-RNG parity instrument
/// (the settle state plus the whole stage opening signature).&lt;/summary&gt;
```

```
        public string Fingerprint()
        {
            var intake = _stage.Intake;
            var sb = new StringBuilder();
            sb.Append("span").Append(_spanIndex).Append('@')
              .Append(_spanSeconds.ToString("0.#", CultureInfo.InvariantCulture)).Append('&gt;')
              .Append(_simSeconds.ToString("0.#", CultureInfo.InvariantCulture));
            sb.Append("|tide").Append(_tideArrivals);
            sb.Append("|earn").Append(_earnHc);
            sb.Append("|armed").Append(_armed ? 1 : 0).Append(_claimed ? "C" : "-");
            sb.Append("|queue").Append(intake.QueueCount).Append(',').Append(intake.ArrivalTotal)
              .Append(',').Append(intake.PatientTotal).Append(',').Append(intake.CuredTotal);
            sb.Append("|fun").Append(_funnel.Count).Append("|sum").Append(_summary.Count);
            foreach (var s in _summary) sb.Append('#').Append(s);
            sb.Append("||").Append(_stage.OpeningSig());
            return sb.ToString();
        }
    }
}
// HospitalPlaceholder.cs
// HospitalPlaceholder.cs - procedural placeholder faces for Crazy Hospital
// (G09-T004). Art-first iron rule (X027): when ArtRuntime holds the stem the
// real sprite wins, otherwise the face falls back to a procedural render -
// views render either way and no headless gate ever depends on art presence.
// The series accent is mint teal #6ED8C8 (the G09 brand face). The emblem
// family is the GREEN CROSS (the red-cross avoidance law: a fictional green
// cross marks every medical face, never a real emblem).
using System.Collections.Generic;
using Sy.Core;
using UnityEngine;
namespace Sy.CrazyHospital
{
    public static class HospitalPlaceholder
    {
        /// &lt;summary&gt;Series accent: mint teal #6ED8C8 (design-language face).&lt;/summary&gt;
        public static readonly Color AccentMint = new Color(110f / 255f, 216f / 255f, 200f / 255f);
        // ---- rank tints: candy clinic -&gt; cream -&gt; fresh mint -&gt; clinical
        // teal -&gt; sterile steel (the vertical facade warms as the hospital
        // ranks up) ----
        static readonly Color[] RankTints =
        {
            new Color(0.96f, 0.86f, 0.80f),   // R1 candy clinic
            new Color(0.97f, 0.94f, 0.82f),    // R2 warm cream
            new Color(0.78f, 0.94f, 0.88f),    // R3 fresh mint
            new Color(0.62f, 0.82f, 0.86f),    // R4 clinical teal
            new Color(0.52f, 0.62f, 0.72f),    // R5 sterile steel
        };
        // ---- stems (art ships under these keys in S2.5; today they miss -&gt; fallback) ----
        public const string FacadeStem = "g09_facade";
        public const string PatientStem = "g09_patient";
```

```
public const string DoctorStem = "g09_doctor";
public const string DeptStem = "g09_dept";
public const string EventCardStem = "g09_eventcard";
public const string PanelStem = "g09_panel";
static readonly Dictionary&lt;string, Sprite&gt; _cache = new Dictionary&lt;string, Sprite&gt;();
/// &lt;summary&gt;True when real art ships under the stem (S2.5 face - the
/// placeholder gates branch on this so art arrival never breaks them).&lt;/summary&gt;
public static bool ArtPresent(string stem) =&gt; ArtRuntime.Get(stem) != null;
/// &lt;summary&gt;The shipped art sprite for a stem (null on the placeholder
/// path) - editor gates compare against this without touching Sy.Core.&lt;/summary&gt;
public static Sprite ArtSprite(string stem) =&gt; ArtRuntime.Get(stem);
/// &lt;summary&gt;Hospital facade tinted by rank (1-5, out-of-range folds
/// to the R1 candy face). The vertical growth roll face. Key law: the
/// shipped catalog stems carry the _r rank suffix (art book sec7,
/// g09_facade_r1..r5) - a bare _&amp;lt;rank&amp;gt; probe silently misses the
/// art and falls back to the tint block (the C1081 wiring bug this
/// resolver must never reintroduce; ArtCheck S3 owns the law).&lt;/summary&gt;
public static Sprite Facade(int rank)
{
    int i = rank - 1;
    if (i &lt; 0 || i &gt;= RankTints.Length) i = 0;
    return Face(FacadeStem + "_r" + (i + 1), RankTints[i], 64);
}
public static Sprite Patient() =&gt; Face(PatientStem, new Color(0.92f, 0.80f, 0.88f), 32);
public static Sprite Doctor() =&gt; Face(DoctorStem, new Color(0.80f, 0.88f, 0.95f), 32);
/// &lt;summary&gt;The department face: art-first, else a green-cross block
/// (the fictional emblem, red-cross avoidance law).&lt;/summary&gt;
public static Sprite Dept()
{
    Sprite art = ArtRuntime.Get(DeptStem);
    if (art != null) return art;
    Sprite cached;
    if (_cache.TryGetValue(DeptStem, out cached) &amp;&amp; cached != null) return cached;
    Sprite made = ProceduralCross(32, AccentMint, new Color(0.94f, 0.98f, 0.97f));
    _cache[DeptStem] = made;
    return made;
}
public static Sprite EventCard() =&gt; Face(EventCardStem, new Color(0.95f, 0.90f, 0.66f), 48);
public static Sprite Panel() =&gt; Face(PanelStem, new Color(0.14f, 0.18f, 0.20f, 0.92f), 64);
/// &lt;summary&gt;Art-first with null-safe procedural fallback (X027 iron rule).&lt;/summary&gt;
public static Sprite Face(string stem, Color color, int px)
{
    Sprite art = ArtRuntime.Get(stem);
    if (art != null) return art;
    string key = stem + "#" + px;
    Sprite cached;
    if (_cache.TryGetValue(key, out cached) &amp;&amp; cached != null) return cached;
    Sprite made = ProceduralSprite(color, px);
    _cache[key] = made;
    return made;
```

```
        }
        /// &lt;summary&gt;Procedural color block: an uncompressed readable px*px square.&lt;/summary&gt;
        public static Sprite ProceduralSprite(Color color, int px)
        {
            var tex = new Texture2D(px, px, TextureFormat.RGBA32, false);
            var pixels = new Color[px * px];
            for (int i = 0; i &lt; pixels.Length; i++) pixels[i] = color;
            tex.SetPixels(pixels);
            tex.Apply(false, false);
            tex.name = "sy_procedural_" + px;
            var sprite = Sprite.Create(tex, new Rect(0, 0, px, px), new Vector2(0.5f, 0.5f), px);
            sprite.name = tex.name;
            return sprite;
        }
        /// &lt;summary&gt;The green-cross procedural emblem: a plus-shaped cross in
        /// the accent color on a soft block (the fictional medical mark - a
        /// pure pixel draw, stable forever for gate pixel reads).&lt;/summary&gt;
        public static Sprite ProceduralCross(int px, Color cross, Color background)
        {
            var tex = new Texture2D(px, px, TextureFormat.RGBA32, false);
            var pixels = new Color[px * px];
            int arm = px / 8;                 // arm thickness (&gt;=1 on any px &gt;= 8)
            if (arm &lt; 1) arm = 1;
            int mid = px / 2;
            for (int y = 0; y &lt; px; y++)
            {
                for (int x = 0; x &lt; px; x++)
                {
                    bool inCross = (x &gt;= mid - arm &amp;&amp; x &lt;= mid + arm - 1) || (y &gt;= mid - arm &amp;&amp;
                    pixels[y * px + x] = inCross ? cross : background;
                }
            }
            tex.SetPixels(pixels);
            tex.Apply(false, false);
            tex.name = "sy_green_cross_" + px;
            var sprite = Sprite.Create(tex, new Rect(0, 0, px, px), new Vector2(0.5f, 0.5f), px);
            sprite.name = tex.name;
            return sprite;
        }
        /// &lt;summary&gt;The rank tint for a hospital rank (1-5) - the visual
        /// progress face for checks.&lt;/summary&gt;
        public static Color TintForRank(int rank)
        {
            int i = rank - 1;
            if (i &lt; 0 || i &gt;= RankTints.Length) i = 0;
            return RankTints[i];
        }
    }
}
// HospitalRank.cs
```

```
// HospitalRank.cs - the rank-review gate engine (G09-T008, SYS05/SYS06).
// The five-rank ladder R1-R5 walks four dual-track gates (numeric pack
// section three): the hard metrics (departments / cure rate / reputation)
// AND the inspection score must hold together - a promotion is a new
// gameplay ticket, never a wall. A failed review only arms the
// rectification comedy (the review expert rides the catapult into the
// hospital's own ward) and re-arms the next review window at the frozen
// inspection cadence (180s, the events-engine cycle - zero new numbers).
// The settlement face is the frozen closed set: pass swings reputation
// +3, fail -3 (HospitalData.ReviewRepSwing); the E01 review bonus folds
// +3 into the score, E04 folds +-2 (the events engine already books
// both). The reputation band is the second numeric line (section four):
// band changes book their visible face flags (banner growth at the top
// band, bad-review comedy at the bottom) plus the evt_band_change
// funnel line - the street flow itself already rides the frozen
// reputation function (TrafficAt), so the band face never re-models it.
// The engine keeps its OWN clock and funnel; the opening signature of
// the stage stays byte-identical until a review resolves (identity
// law). A resolved promotion folds the swap into both hosts inside the
// settle: the intake economy row (PromoteToRank) and the events stage
// pool (SetRank) - one verdict, three faces agree (the T008 host
// wiring). Pure C#, no MonoBehaviour.
using System;
using System.Collections.Generic;
namespace Sy.CrazyHospital
{
    /// &lt;summary&gt;One review settlement (the dual-track verdict).&lt;/summary&gt;
    public struct ReviewVerdict
    {
        public bool pass;
        public bool hardOk;
        public bool scoreOk;
    }
    /// &lt;summary&gt;SYS05/SYS06: the rank-review gate + reputation band
    /// engine. The host wires Advance(dt); headless gates drive the
    /// static solver and the gate faces directly.&lt;/summary&gt;
    public sealed class HospitalRank
    {
        // ---- the frozen review laws (closed set, zero new numbers) ----
        public const double ReviewCooldownSeconds = HospitalEvents.CadenceSeconds; // re-review = the next inspection cy
        public const int ScoreBonus = HospitalData.ReviewRepSwing;               // the E01 pass folds +3 into the score
        readonly HospitalIntake _intake;
        readonly HospitalEvents _events;
        readonly List&lt;string&gt; _funnel = new List&lt;string&gt;();
        double _t;
        double _cooldownUntil = -1;     // sim time until the next review window opens (-1 = open)
        int _rank = 1;
        int _promotions;
        int _failedReviews;
        int _lastVerdictRank;           // rank the last review ran under (0 = never)
```

```
bool _lastPass;
bool _ceremonyArmed;            // pass -&gt; the stamp ceremony (fireworks + banner rain)
bool _remediationArmed;         // fail -&gt; the rectification comedy face
bool _badReviewComedyArmed;     // band entry into rep_bad
bool _bannerGrowthArmed;        // band entry into rep_banner
string _bandId = "";
public HospitalRank(HospitalIntake intake, HospitalEvents events, int startRank = 1)
{
    _intake = intake;
    _events = events;
    _rank = startRank &lt; 1 ? 1 : startRank;
    _bandId = BandFace().id;
}
// ---- live faces ----
public double SimTime =&gt; _t;
public int Rank =&gt; _rank;
public int Promotions =&gt; _promotions;
public int FailedReviews =&gt; _failedReviews;
public int LastVerdictRank =&gt; _lastVerdictRank;
public bool LastPass =&gt; _lastPass;
public bool CeremonyArmed =&gt; _ceremonyArmed;
public bool RemediationArmed =&gt; _remediationArmed;
public bool BadReviewComedyArmed =&gt; _badReviewComedyArmed;
public bool BannerGrowthArmed =&gt; _bannerGrowthArmed;
public string BandId =&gt; _bandId;
public IReadOnlyList&lt;string&gt; Funnel =&gt; _funnel;
public bool ReviewWindowOpen =&gt; _t &gt;= _cooldownUntil;
/// &lt;summary&gt;The gate the current rank faces (null at R5: the
/// ladder is complete, the top-grade stamp is the ending face).&lt;/summary&gt;
public ReviewGateDef CurrentGate
{
    get
    {
        foreach (var g in HospitalData.Gates())
            if (g.fromRank == _rank) return g;
        return null;
    }
}
/// &lt;summary&gt;The live inspection score: the events-engine delta
/// plus the armed E01 review bonus (the closed set: E04 +-2,
/// E01 pass +3).&lt;/summary&gt;
public int InspectScore
    =&gt; (_events != null ? _events.InspectScoreDelta : 0)
     + (_events != null &amp;&amp; _events.ReviewBonusPending ? ScoreBonus : 0);
/// &lt;summary&gt;Live hard-metric faces (the host already owns the
/// sim; the solver reads, never models).&lt;/summary&gt;
public int DeptsOf() =&gt; _intake != null ? _intake.OpenSlots : 0;
public int ReputationOf() =&gt; _intake != null ? _intake.Reputation : 0;
public int CureRatePctOf()
{
```

```
    if (_intake == null || _intake.PatientTotal &lt;= 0) return 0;
    return _intake.CuredTotal * 100 / _intake.PatientTotal;
}
/// &lt;summary&gt;The reputation band (the frozen section-four
/// partition; the street flow rides TrafficAt, never re-model).&lt;/summary&gt;
public ReputationBand BandFace()
    =&gt; _intake != null ? HospitalData.BandFor(_intake.Reputation)
        : HospitalData.BandFor(HospitalData.ReputationMin);
/// &lt;summary&gt;Static dual-track solver (the headless-gate face:
/// verdict math is pure, no runtime state).&lt;/summary&gt;
public static ReviewVerdict Solve(ReviewGateDef gate, int depts, int cureRatePct, int reputation, int inspectSco
{
    ReviewVerdict v;
    v.hardOk = gate != null
        &amp;&amp; depts &gt;= gate.minDepts
        &amp;&amp; cureRatePct &gt;= gate.minCureRatePct
        &amp;&amp; reputation &gt;= gate.minReputation;
    v.scoreOk = inspectScore &gt;= 0;   // a net non-negative inspection record
    v.pass = gate != null &amp;&amp; v.hardOk &amp;&amp; v.scoreOk;   // dual-track law: both tracks together
    return v;
}
/// &lt;summary&gt;Requests a review against the current gate: refuses
/// while the cooldown window holds (fail = the next inspection
/// cycle) or at R5. On resolve books the settlement (rep swing
/// +-3), the ceremony / remediation flags and the evt_promote /
/// evt_inspect funnel lines. A pass promotes the ladder AND folds
/// the swap into both hosts inside the settle: the intake economy
/// row and the events stage pool read the new rank from the next
/// tick on (one verdict, three faces agree).&lt;/summary&gt;
public ReviewVerdict RequestReview()
{
    var gate = CurrentGate;
    if (gate == null || !ReviewWindowOpen) { var none = new ReviewVerdict(); return none; }
    ReviewVerdict v = Solve(gate, DeptsOf(), CureRatePctOf(), ReputationOf(), InspectScore);
    _lastVerdictRank = _rank;
    _lastPass = v.pass;
    if (_intake != null)
        _intake.SetReputation(_intake.Reputation + (v.pass ? HospitalData.ReviewRepSwing : -HospitalData.ReviewR
    if (v.pass)
    {
        _rank = gate.toRank;
        if (_intake != null) _intake.PromoteToRank(gate.toRank);   // the host fold: the economy row swaps with
        if (_events != null) _events.SetRank(gate.toRank);         // the host fold: the stage pool widens with
        _promotions++;
        _ceremonyArmed = true;
        _funnel.Add("evt_promote@r" + gate.toRank + ":" + gate.reviewEventId);
    }
    else
    {
        _failedReviews++;
```

```
        _remediationArmed = true;
        _cooldownUntil = _t + ReviewCooldownSeconds;   // re-review = the next inspection cycle
        _funnel.Add("evt_inspect@r" + gate.fromRank + ":" + gate.reviewEventId);
    }
    return v;
}
/// &lt;summary&gt;Advances the engine clock and tracks the reputation
/// band face: a band change books the visible comedy flags and
/// the evt_band_change funnel line (banner growth at the top
/// band, bad-review comedy at the bottom - the two-way face of
/// the second numeric line).&lt;/summary&gt;
public void Advance(double dt)
{
    if (dt &lt;= 0) return;
    _t += dt;
    var band = BandFace();
    if (band.id != _bandId)
    {
        _bandId = band.id;
        _badReviewComedyArmed = band.id == "rep_bad";
        _bannerGrowthArmed = band.id == "rep_banner";
        _funnel.Add("evt_band_change@" + band.id);
    }
}
/// &lt;summary&gt;Gate faces (headless checks drive the engine without
/// a live intake/events pair).&lt;/summary&gt;
public void ArmForGate(int rank, int reputation)
{
    _rank = rank &lt; 1 ? 1 : rank;
    if (_intake != null) _intake.SetReputation(reputation);
    _cooldownUntil = -1;
}
/// &lt;summary&gt;Canonical fingerprint: the zero-RNG parity
/// instrument (same script = same bytes).&lt;/summary&gt;
public string Fingerprint()
{
    var sb = new System.Text.StringBuilder();
    sb.Append("t=").Append(_t.ToString("0.###"));
    sb.Append("|rank=").Append(_rank);
    sb.Append("|promo=").Append(_promotions);
    sb.Append("|fail=").Append(_failedReviews);
    sb.Append("|last=").Append(_lastVerdictRank).Append('/').Append(_lastPass ? 1 : 0);
    sb.Append("|score=").Append(InspectScore);
    sb.Append("|cd=").Append(_cooldownUntil.ToString("0.###"));
    sb.Append("|flags=").Append(_ceremonyArmed ? 1 : 0).Append(_remediationArmed ? 1 : 0)
      .Append(_badReviewComedyArmed ? 1 : 0).Append(_bannerGrowthArmed ? 1 : 0);
    sb.Append("|band=").Append(_bandId);
    sb.Append("|fun=").Append(_funnel.Count);
    return sb.ToString();
}
```

```
    }
}
// HospitalSave.cs
// HospitalSave.cs - the save system (G09-T014, G06SaveSystem / G12SaveSystem
// paradigm): single-key JSON persistence through the Sy.Core save service.
// Local storage is the only authority (red line 12, zero-server doctrine).
//
// Faces:
// - HospitalProfile: the save payload model. Only earned state is stored -
//   gold, rank, reputation, the department slot choice history (replayable),
//   the doctor card collection in hire order, the three-volume album rows,
//   research spend and the booked event chain flags. Duty roster state is
//   NEVER stored: it derives from the collection + open slot count on every
//   SyncRoster call, so a drifted duty row is structurally impossible
//   (derive-live zero-drift doctrine, G12 unlock-face sibling).
// - HospitalSaveSystem: four-state load (NewGame / Loaded / Migrated /
//   CorruptedReset). Version-migration skeleton: a payload without the
//   version key migrates by filling defaults for missing fields; a payload
//   from a newer build degrades to a fresh profile instead of guessing at
//   its schema. Corrupted rows drop with warnings, never fatal - a save can
//   brick a run only by refusing to load, never by throwing.
// - HospitalSaveSchema: editor-check verdict faces (pure C#, zero platform
//   coupling beyond ISaveService itself) so the Editor check rides these
//   faces instead of naming Sy.Core types directly (G12SaveSchema
//   paradigm). Faces mirror the save DoD: round-trip + four states +
//   migration defaults + tamper clamps + NG+ carry reconciliation against
//   HospitalEnding.SolveCarry / ApplyCarry.
using System;
using System.Collections.Generic;
using Sy.Core;
namespace Sy.CrazyHospital
{
    /// &lt;summary&gt;Outcome of a save-slot load (G06 four-state paradigm).&lt;/summary&gt;
    public enum HospitalLoadStatus
    {
        NewGame,         // slot empty: fresh dean, nothing written yet
        Loaded,          // current-version payload applied as-is
        Migrated,        // older payload upgraded in place (defaults filled)
        CorruptedReset   // unreadable or future-version payload: profile reset
    }
    /// &lt;summary&gt;
    /// The G09 save payload (task T014): every earned face of the S2
    /// runtime in one serializable bag. Slot choices are the replayable
    /// history (pick WHAT opened, in open order) - the slot count, the
    /// last choice and the duty roster all derive from it live.
    /// &lt;/summary&gt;
    public sealed class HospitalProfile
    {
        public int GoldHc;                          // the wallet (gold hundredths)
        public int Rank = 1;                        // R1..R5 ladder state
```

```
    public int Reputation;                      // 0-100 street fame
    public List&lt;int&gt; SlotChoices = new List&lt;int&gt;();  // department choice history
    public int PullCount;                       // doctor card draws taken
    public int DupeTotal;                       // duplicate cards folded
    public int ResearchSpendHc;                 // gold sunk into wonder drugs
    public List&lt;string&gt; DoctorIds = new List&lt;string&gt;();  // hire-order collection
    public List&lt;string&gt; AlbumIds = new List&lt;string&gt;();    // the three-volume unlocked rows
    public List&lt;string&gt; EventFlags = new List&lt;string&gt;();  // booked event chain flags
    /// &lt;summary&gt;Derived live from the replay history, never stored twice.&lt;/summary&gt;
    public int OpenSlots =&gt; SlotChoices.Count;
}
/// &lt;summary&gt;
/// Single-key JSON persistence through the Sy.Core save service
/// (local authority, red line 12). All reads clamp to the frozen
/// schema laws (HospitalData) - a tampered payload can only ever
/// produce a schema-legal profile.
/// &lt;/summary&gt;
public static class HospitalSaveSystem
{
    public const string Slot = "g09_main";
    public const string VersionKey = "g09v";
    public const int CurrentVersion = 1;
    /// &lt;summary&gt;Serialize the profile into the single-key JSON payload.&lt;/summary&gt;
    public static string Serialize(HospitalProfile profile)
    {
        if (profile == null) throw new ArgumentNullException(nameof(profile));
        var choices = new List&lt;object&gt;();
        if (profile.SlotChoices != null)
            foreach (var c in profile.SlotChoices) choices.Add((long)c);
        var docs = new List&lt;object&gt;();
        if (profile.DoctorIds != null)
            foreach (var id in profile.DoctorIds) docs.Add(id);
        var albums = new List&lt;object&gt;();
        if (profile.AlbumIds != null)
            foreach (var id in profile.AlbumIds) albums.Add(id);
        var flags = new List&lt;object&gt;();
        if (profile.EventFlags != null)
            foreach (var id in profile.EventFlags) flags.Add(id);
        var data = new Dictionary&lt;string, object&gt;
        {
            { VersionKey, (long)CurrentVersion },
            { "gold", (long)profile.GoldHc },
            { "rank", (long)profile.Rank },
            { "rep", (long)profile.Reputation },
            { "choices", choices },
            { "pulls", (long)profile.PullCount },
            { "dupes", (long)profile.DupeTotal },
            { "rshc", (long)profile.ResearchSpendHc },
            { "docs", docs },
            { "albums", albums },
```

```
            { "flags", flags },
        };
        return MiniJson.Serialize(data);
    }
    /// &lt;summary&gt;Write the profile through the save service (local authority).&lt;/summary&gt;
    public static void Persist(ISaveService save, HospitalProfile profile, string slot = Slot)
    {
        if (save == null) return;
        save.Save(slot, Serialize(profile));
    }
    /// &lt;summary&gt;
    /// Load the slot into the given profile. Never throws on bad data:
    /// corrupted or future-version payloads reset the profile to fresh
    /// defaults. NewGame and CorruptedReset both reset deterministically
    /// first, so a stale caller-side profile never survives either fresh
    /// outcome.
    /// &lt;/summary&gt;
    public static HospitalLoadStatus LoadInto(ISaveService save, HospitalProfile profile, string slot = Slot)
    {
        if (save == null) throw new ArgumentNullException(nameof(save));
        if (profile == null) throw new ArgumentNullException(nameof(profile));
        string raw;
        try
        {
            raw = save.Load(slot);
        }
        catch (Exception e)
        {
            // the service itself chokes on a corrupted payload - degrade, never crash the boot
            Logger.Warn("G09.Save", "corrupt payload at service level, resetting slot: " + e.Message);
            return Fresh(profile);
        }
        if (string.IsNullOrEmpty(raw))
        {
            ResetProfile(profile);
            return HospitalLoadStatus.NewGame;
        }
        Dictionary&lt;string, object&gt; data;
        try
        {
            data = MiniJson.Deserialize(raw) as Dictionary&lt;string, object&gt;;
        }
        catch (Exception e)
        {
            Logger.Warn("G09.Save", "corrupt payload, resetting slot: " + e.Message);
            return Fresh(profile);
        }
        if (data == null)
        {
            Logger.Warn("G09.Save", "non-dict payload, resetting slot");
```

```
            return Fresh(profile);
        }
        int version = ReadInt(data, VersionKey, 0);
        if (version &gt; CurrentVersion)
        {
            Logger.Warn("G09.Save", "future version " + version + " &gt; " + CurrentVersion + ", resetting slot");
            return Fresh(profile);
        }
        profile.GoldHc = ClampMin(ReadInt(data, "gold", 0), 0);            // wallet law: spend refuses before negat
        profile.Rank = Clamp(ReadInt(data, "rank", 1), 1, HospitalData.RankCount);
        profile.Reputation = Clamp(ReadInt(data, "rep", 0),
            HospitalData.ReputationMin, HospitalData.ReputationMax);
        profile.SlotChoices = ReadInts(data, "choices");
        profile.PullCount = ClampMin(ReadInt(data, "pulls", 0), 0);
        profile.DupeTotal = ClampMin(ReadInt(data, "dupes", 0), 0);
        profile.ResearchSpendHc = ClampMin(ReadInt(data, "rshc", 0), 0);
        profile.DoctorIds = ReadIds(data, "docs");
        profile.AlbumIds = ReadIds(data, "albums");
        profile.EventFlags = ReadIds(data, "flags");
        return version &lt; CurrentVersion ? HospitalLoadStatus.Migrated : HospitalLoadStatus.Loaded;
    }
    static HospitalLoadStatus Fresh(HospitalProfile profile)
    {
        ResetProfile(profile);
        return HospitalLoadStatus.CorruptedReset;
    }
    static void ResetProfile(HospitalProfile profile)
    {
        profile.GoldHc = 0;
        profile.Rank = 1;
        profile.Reputation = 0;
        profile.SlotChoices = new List&lt;int&gt;();
        profile.PullCount = 0;
        profile.DupeTotal = 0;
        profile.ResearchSpendHc = 0;
        profile.DoctorIds = new List&lt;string&gt;();
        profile.AlbumIds = new List&lt;string&gt;();
        profile.EventFlags = new List&lt;string&gt;();
    }
    static int Clamp(int v, int lo, int hi) =&gt; v &lt; lo ? lo : (v &gt; hi ? hi : v);
    static int ClampMin(int v, int lo) =&gt; v &lt; lo ? lo : v;
    static int ReadInt(Dictionary&lt;string, object&gt; data, string key, int fallback)
    {
        if (!data.TryGetValue(key, out var value) || value == null) return fallback;
        try { return Convert.ToInt32(Convert.ToDouble(value)); }
        catch { return fallback; }
    }
    static List&lt;int&gt; ReadInts(Dictionary&lt;string, object&gt; data, string key)
    {
        var list = new List&lt;int&gt;();
```

```
            if (!data.TryGetValue(key, out var value) || !(value is System.Collections.IEnumerable rows)) return list;
            foreach (var row in rows)
            {
                int v;
                try { v = Convert.ToInt32(Convert.ToDouble(row)); }
                catch
                {
                    Logger.Warn("G09.Save", "non-int choice row, dropped");
                    continue;
                }
                if (v &lt; 0)
                {
                    Logger.Warn("G09.Save", "negative choice row, dropped");
                    continue;
                }
                list.Add(v);
            }
            return list;
        }
        /// &lt;summary&gt;Id rows: null / empty / duplicate entries drop with a
        /// warning (a save can never inflate a collection face).&lt;/summary&gt;
        static List&lt;string&gt; ReadIds(Dictionary&lt;string, object&gt; data, string key)
        {
            var list = new List&lt;string&gt;();
            var seen = new HashSet&lt;string&gt;();
            if (!data.TryGetValue(key, out var value) || !(value is System.Collections.IEnumerable rows)) return list;
            foreach (var row in rows)
            {
                string id = row as string;
                if (string.IsNullOrEmpty(id))
                {
                    Logger.Warn("G09.Save", key + " row without id, dropped");
                    continue;
                }
                if (!seen.Add(id))
                {
                    Logger.Warn("G09.Save", key + " duplicate row, dropped: " + id);
                    continue;
                }
                list.Add(id);
            }
            return list;
        }
    }
    /// &lt;summary&gt;
    /// Editor-check verdict faces for the save domain (G12SaveSchema
    /// paradigm): pure C#, EditMode-pinnable, mirroring the save DoD in
    /// machine-readable form so the SaveCheck gate owns round-trip + four
    /// states + migration defaults + tamper clamps + the NG+ carry
    /// reconciliation against HospitalEnding.
```

```
/// &lt;/summary&gt;
public static class HospitalSaveSchema
{
    /// &lt;summary&gt;In-memory ISaveService stub (check path only, zero platform coupling).&lt;/summary&gt;
    sealed class MemorySaveService : ISaveService
    {
        readonly Dictionary&lt;string, string&gt; slots = new Dictionary&lt;string, string&gt;();
        public void Save(string slot, string payloadJson) { slots[slot] = payloadJson; }
        public string Load(string slot)
        {
            string v;
            return slots.TryGetValue(slot, out v) ? v : null;
        }
        public SyncResult CloudSync() { return new SyncResult(false, null); }
    }
    static HospitalProfile MakeSeededProfile()
    {
        return new HospitalProfile
        {
            GoldHc = 12345,
            Rank = 3,
            Reputation = 62,
            SlotChoices = new List&lt;int&gt; { 0, 2, 1 },
            PullCount = 7,
            DupeTotal = 2,
            ResearchSpendHc = 350,
            DoctorIds = new List&lt;string&gt; { "doc_threesec", "doc_wangdali" },
            AlbumIds = new List&lt;string&gt; { "dis_hiccup_bug", "drug_hiccup_cola" },
            EventFlags = new List&lt;string&gt; { HospitalEvents.ChainReviewBonus, HospitalEvents.ChainBanner },
        };
    }
    /// &lt;summary&gt;Serialize - persist - reload keeps every field and re-serializes identically.&lt;/summary&gt;
    public static bool RoundTripPreservesPayload()
    {
        var save = new MemorySaveService();
        var seeded = MakeSeededProfile();
        string once = HospitalSaveSystem.Serialize(seeded);
        HospitalSaveSystem.Persist(save, seeded);
        var back = new HospitalProfile();
        if (HospitalSaveSystem.LoadInto(save, back) != HospitalLoadStatus.Loaded) return false;
        if (HospitalSaveSystem.Serialize(back) != once) return false;
        return back.GoldHc == 12345 &amp;&amp; back.Rank == 3 &amp;&amp; back.Reputation == 62
            &amp;&amp; back.OpenSlots == 3 &amp;&amp; back.PullCount == 7 &amp;&amp; back.DupeTotal == 2
            &amp;&amp; back.ResearchSpendHc == 350
            &amp;&amp; back.DoctorIds.Count == 2 &amp;&amp; back.DoctorIds[0] == "doc_threesec"
            &amp;&amp; back.AlbumIds.Count == 2 &amp;&amp; back.AlbumIds[0] == "dis_hiccup_bug"
            &amp;&amp; back.EventFlags.Count == 2;
    }
    /// &lt;summary&gt;Empty slot = NewGame, and the reset wipes any stale caller-side rows.&lt;/summary&gt;
    public static bool StateNewGameOnEmptySlot()
```

```
{
    var save = new MemorySaveService();
    var stale = MakeSeededProfile();
    if (HospitalSaveSystem.LoadInto(save, stale) != HospitalLoadStatus.NewGame) return false;
    return stale.GoldHc == 0 &amp;&amp; stale.Rank == 1 &amp;&amp; stale.OpenSlots == 0
        &amp;&amp; stale.DoctorIds.Count == 0 &amp;&amp; stale.AlbumIds.Count == 0 &amp;&amp; stale.EventFlags.C
}
/// &lt;summary&gt;Version-less payload (pre-version build) migrates by filling defaults.&lt;/summary&gt;
public static bool StateMigratedOnVersionlessPayload()
{
    var save = new MemorySaveService();
    save.Save(HospitalSaveSystem.Slot,
        "{\"gold\":900,\"rank\":2,\"rep\":40,\"choices\":[0],\"docs\":[\"doc_wangdali\"],\"albums\":[],\"flags\"
    var profile = new HospitalProfile();
    if (HospitalSaveSystem.LoadInto(save, profile) != HospitalLoadStatus.Migrated) return false;
    // the missing keys (pulls / dupes / rshc) migrate to their defaults
    return profile.GoldHc == 900 &amp;&amp; profile.Rank == 2 &amp;&amp; profile.Reputation == 40
        &amp;&amp; profile.OpenSlots == 1 &amp;&amp; profile.PullCount == 0 &amp;&amp; profile.DupeTotal == 0
        &amp;&amp; profile.ResearchSpendHc == 0
        &amp;&amp; profile.DoctorIds.Count == 1 &amp;&amp; profile.DoctorIds[0] == "doc_wangdali";
}
/// &lt;summary&gt;Unparseable payload degrades to a fresh profile instead of crashing the boot.&lt;/summary&gt;
public static bool StateCorruptedResetOnGarbage()
{
    var save = new MemorySaveService();
    save.Save(HospitalSaveSystem.Slot, "{ not json at all");
    var stale = MakeSeededProfile();
    if (HospitalSaveSystem.LoadInto(save, stale) != HospitalLoadStatus.CorruptedReset) return false;
    return stale.GoldHc == 0 &amp;&amp; stale.OpenSlots == 0 &amp;&amp; stale.DoctorIds.Count == 0;
}
/// &lt;summary&gt;A payload from a newer build resets rather than guessing at its schema.&lt;/summary&gt;
public static bool StateCorruptedResetOnFutureVersion()
{
    var save = new MemorySaveService();
    save.Save(HospitalSaveSystem.Slot, "{\"g09v\":99,\"gold\":1}");
    var profile = new HospitalProfile();
    return HospitalSaveSystem.LoadInto(save, profile) == HospitalLoadStatus.CorruptedReset
        &amp;&amp; profile.GoldHc == 0;
}
/// &lt;summary&gt;A tampered payload can only ever produce a schema-legal
/// profile: the frozen clamps hold and collection rows dedupe.&lt;/summary&gt;
public static bool TamperClampsToSchemaLaws()
{
    var save = new MemorySaveService();
    save.Save(HospitalSaveSystem.Slot,
        "{\"g09v\":1,\"gold\":-500,\"rank\":9,\"rep\":250,\"choices\":[-1,2],\"pulls\":-4,"
        + "\"docs\":[\"doc_wangdali\",\"doc_wangdali\",\"\"],\"albums\":[\"dis_monday\"],\"flags\":[]}");
    var profile = new HospitalProfile();
    if (HospitalSaveSystem.LoadInto(save, profile) != HospitalLoadStatus.Loaded) return false;
    return profile.GoldHc == 0
```

```
        &amp;&amp; profile.Rank == HospitalData.RankCount
        &amp;&amp; profile.Reputation == HospitalData.ReputationMax
        &amp;&amp; profile.OpenSlots == 1 &amp;&amp; profile.SlotChoices[0] == 2
        &amp;&amp; profile.PullCount == 0
        &amp;&amp; profile.DoctorIds.Count == 1 &amp;&amp; profile.DoctorIds[0] == "doc_wangdali";
}
/// &lt;summary&gt;NG+ carry reconciliation (task DoD: the inherit face
/// matches HospitalEnding): the carry pack folded from the saved
/// profile's faces keeps gold 20% floored and the album rows, rank
/// and event flags reset by simply NOT being carried, and applying
/// the pack onto a fresh album replants the same ids with zero drift.&lt;/summary&gt;
public static bool NgPlusCarryMatchesEndingFaces()
{
    var save = new MemorySaveService();
    var seeded = MakeSeededProfile();
    HospitalSaveSystem.Persist(save, seeded);
    var back = new HospitalProfile();
    if (HospitalSaveSystem.LoadInto(save, back) != HospitalLoadStatus.Loaded) return false;
    // fresh run faces: album replanted from the saved rows, doctor desk null
    // (the doctor id leg rides the profile round-trip, the desk's own
    // InheritFrom idempotence is DoctorCheck territory)
    var freshAlbum = new HospitalAlbum(null);
    var carry = HospitalEnding.SolveCarry(back.GoldHc, AlbumFromProfile(back), null);
    if (carry.GoldCarryHc != HospitalEnding.SolveGoldCarryHc(back.GoldHc)) return false;
    if (carry.GoldCarryHc != back.GoldHc * HospitalEnding.GoldCarryPercent / 100) return false;
    if (carry.DoctorIds == null || carry.DoctorIds.Count != 0) return false;
    var ending = new HospitalEnding();
    ending.ApplyCarry(carry, freshAlbum, null);
    // the replanted album rows match the saved rows exactly (zero drift)
    if (freshAlbum.Unlocked.Count != back.AlbumIds.Count) return false;
    for (int i = 0; i &lt; back.AlbumIds.Count; i++)
        if (freshAlbum.Unlocked[i] != back.AlbumIds[i]) return false;
    // the ending books its one carry line, and the once-guard facade face stays untouched
    if (ending.FacadeLit) return false;
    // the fresh run's own save profile is a new-game identity: rank / flags never carry
    var freshProfile = new HospitalProfile();
    if (freshProfile.Rank != 1 || freshProfile.EventFlags.Count != 0) return false;
    return true;
}
/// &lt;summary&gt;Duty roster state derives live from the collection +
/// slot count (SyncRoster), never from the save payload - a drifted
/// duty row is structurally impossible.&lt;/summary&gt;
public static bool DutyRosterDerivesZeroDrift()
{
    var desk = new HospitalDoctors(null, null, () =&gt; "gate_day");
    desk.InheritFrom(new List&lt;string&gt; { "doc_threesec", "doc_wangdali" });
    desk.SyncRoster(1);
    if (desk.Roster.Count != 1) return false;
    desk.SyncRoster(4);
    if (desk.Roster.Count != 2) return false;
```

```
            // the save payload carries no roster field at all: the serialized
            // seed contains no duty face, only the collection ids
            string json = HospitalSaveSystem.Serialize(MakeSeededProfile());
            return json.Contains("docs") &amp;&amp; !json.Contains("duty") &amp;&amp; !json.Contains("roster");
        }
        static HospitalAlbum AlbumFromProfile(HospitalProfile profile)
        {
            var album = new HospitalAlbum(null);
            if (profile != null &amp;&amp; profile.AlbumIds != null) album.InheritFrom(profile.AlbumIds);
            return album;
        }
    }
}
// HospitalShareCard.cs
// HospitalShareCard.cs - the top-grade stamp certificate share card
// (G09-T011 prefix, SYS08 / R5 ending face): the brag card a stamped
// dean shows off - the grade title, four stat lines, the honor-wall
// photo row and the challenge footer - composed through the shared
// AutoCardComposer face (Sy.Social; template key g09_sharecard:
// missing art falls back to the procedural dark card, headless stays
// green). Every delivered share books evt_album_share; a compose that
// never lands books nothing (the funnel only ever counts real shares).
// The honor-wall law is closed-set: one stamp cell (tier = rank 1-5)
// plus one cell per album volume (tier = rate/20 floored: 0 empty,
// 100% gold). Pure C# + the shared composer, zero RNG, zero scene
// state, every copy routed through HospitalTexts keys.
using System.Collections.Generic;
using Sy.Social;
using UnityEngine;
namespace Sy.CrazyHospital
{
    public sealed class HospitalShareCard
    {
        public const string TemplateKey = "g09_sharecard";
        public const string EventId = "evt_album_share";
        public const string QueryPrefix = "g09";
        public const int WallCells = 4;               // stamp + three album volumes
        public const int PinnedComposeBudgetBytes = 8 * 1024 * 1024;   // canary budget pins the 1080x1440 tier
        readonly List&lt;string&gt; _funnel = new List&lt;string&gt;();
        public IReadOnlyList&lt;string&gt; Funnel =&gt; _funnel;
        /// &lt;summary&gt;Honor-wall tier solver (closed set): rate 0-19 -&gt; 0
        /// (empty cell), each +20% one tier up, 100% = gold tier 5,
        /// out-of-range rates clamp.&lt;/summary&gt;
        public static int SolveWallTier(int ratePct)
            =&gt; Mathf.Clamp(ratePct / 20, 0, 5);
        /// &lt;summary&gt;The collection-depth face of the share payload: the
        /// sum of wall tiers (the query carries it as vols).&lt;/summary&gt;
        public static int VolumeTierSum(int[] volumeRates)
        {
            int n = 0;
```

```
    if (volumeRates != null) for (int i = 0; i &lt; volumeRates.Length &amp;amp;&amp;amp; i &lt; 3; i++) n += SolveWallT
    return n;
}
/// &lt;summary&gt;Pure spec solver (the headless face): title = the
/// grade name of the rank, lines = rep / cure / depts / on-duty,
/// footer = the challenge, wall = stamp + three volume cells in
/// pack order (diseases / doctors / drugs).&lt;/summary&gt;
public static ShareCard SolveSpec(int rank, int reputation, int cureRatePct,
    int depts, int doctorsOnDuty, int[] volumeRates)
{
    rank = Mathf.Clamp(rank, 1, 5);
    var rates = volumeRates ?? new int[0];
    var cells = new CardCell[WallCells];
    cells[0] = new CardCell { Tier = rank, Stack = 1 };
    for (int i = 0; i &lt; 3; i++)
        cells[i + 1] = new CardCell { Tier = rates.Length &gt; i ? SolveWallTier(rates[i]) : 0, Stack = 1 };
    var lines = new[]
    {
        HospitalTexts.EnText(HospitalTexts.KeyStatReputation) + ": " + reputation,
        HospitalTexts.EnText(HospitalTexts.KeyStatCureRate) + ": " + Mathf.Max(0, cureRatePct) + "%",
        HospitalTexts.EnText(HospitalTexts.KeyStatDepts) + ": " + Mathf.Max(0, depts),
        HospitalTexts.EnText(HospitalTexts.KeyDutyRoster) + ": " + Mathf.Max(0, doctorsOnDuty),
    };
    var query = QueryPrefix + "&amp;rank=" + rank + "&amp;rep=" + reputation
        + "&amp;cure=" + Mathf.Max(0, cureRatePct) + "&amp;depts=" + Mathf.Max(0, depts)
        + "&amp;docs=" + Mathf.Max(0, doctorsOnDuty) + "&amp;vols=" + VolumeTierSum(rates);
    var card = AutoCardComposer.Card(TemplateKey,
        HospitalTexts.EnText("rank_" + rank), lines,
        HospitalTexts.EnText(HospitalTexts.KeyShareChallenge), query);
    card.Formation = cells;
    card.FormationCols = WallCells;
    card.FormationRows = 1;
    return card;
}
/// &lt;summary&gt;Live fold face: reads the wired faces and folds into
/// the pure solver (the host owns the sim, this face only reads;
/// null faces fold to the honest zeros face).&lt;/summary&gt;
public static ShareCard BuildSpec(HospitalRank rank, HospitalAlbum album, HospitalDoctors doctors)
{
    var rates = new int[3];
    if (album != null)
        for (int i = 0; i &lt; 3; i++) rates[i] = album.RatePctOf(HospitalAlbum.VolumeIds()[i]);
    return SolveSpec(
        rank != null ? rank.Rank : 1,
        rank != null ? rank.ReputationOf() : 0,
        rank != null ? rank.CureRatePctOf() : 0,
        rank != null ? rank.DeptsOf() : 0,
        doctors != null ? doctors.CollectedCount : 0,
        rates);
}
```

```
        /// &lt;summary&gt;Shares the certificate: builds the spec, composes
        /// the PNG through the shared RT face and books evt_album_share
        /// only on a delivered card (Ok=false composes book nothing).
        /// Returns the compose verdict for the caller to surface.&lt;/summary&gt;
        public ComposeResult Share(HospitalRank rank, HospitalAlbum album, HospitalDoctors doctors,
            int maxPngBytes = ShareCardService.MaxPngBytes)
        {
            var card = BuildSpec(rank, album, doctors);
            var res = new ShareCardService().Compose(card, maxPngBytes);
            if (res != null &amp;&amp; res.Ok)
                _funnel.Add(EventId + "@r" + (rank != null ? rank.Rank : 1) + ":" + res.PngBytes);
            return res;
        }
        /// &lt;summary&gt;Canonical fingerprint: the zero-RNG parity
        /// instrument (same script = same bytes).&lt;/summary&gt;
        public string Fingerprint()
        {
            var sb = new System.Text.StringBuilder();
            sb.Append("fun=").Append(_funnel.Count);
            for (int i = 0; i &lt; _funnel.Count; i++) sb.Append('|').Append(_funnel[i]);
            return sb.ToString();
        }
    }
}
// HospitalStage.cs
// HospitalStage.cs - the opening beat driver (G09-T005).
// Drives HospitalIntake through the frozen red-line-11 opening script and
// stamps every promise on the sim clock: the facade hook owns t0, the first
// department decision lands inside the 3-8s window, the queue head registers
// half a second later and the first coin fires visual+sfx+number on one
// tick, then the first fictional patient portrait with the absurd diagnosis
// line (the first laugh) and the first review preview card land inside the
// 10s window. Pure C# zero-RNG: the same script always fingerprints
// byte-identical (Redline11Check consumes this face).
using System.Collections.Generic;
namespace Sy.CrazyHospital
{
    public enum HospitalPhase
    {
        Hook,          // the facade + the street queue caption
        ClinicOpen,    // the first department decision made, doors open
        FirstCoin,     // the first registration coin landed
        FirstLaughter, // the absurd diagnosis portrait fired (the first laugh)
    }
    public sealed class HospitalStage
    {
        // the frozen opening script (red-line-11 beat anchors)
        public const double DecisionWindowStart = 3.0;   // T+3s: the doors decision face goes live
        public const double FirstLaughAt = 8.5;           // T+8.5s: patient portrait + absurd diagnosis
        public const double ReviewPreviewAt = 9.5;        // T+9.5s: the first review preview card
```

```
readonly HospitalIntake _intake;
readonly HospitalEvents _events;
readonly HospitalRank _rank;   // T008: the rank-review gate rides the opening clock
readonly List&lt;(string id, double t)&gt; _stamps = new List&lt;(string, double)&gt;();
double _t;
bool _deptOpen;
double _deptOpenedAt = -1;
bool _firstCoin;
bool _admitBlocked;   // the registration attempt refused this call: retry on the next Advance (per-call law)
bool _laughDone;
bool _previewDone;
string _firstPatientDiag = "";
int _firstCoinGoldBefore;
int _firstCoinJumpHc;
HospitalStage(HospitalIntake intake)
{
    _intake = intake;
    _events = new HospitalEvents(intake, 1);   // T007: the event engine rides the opening clock (own timeline -
    _rank = new HospitalRank(intake, _events, 1);   // T008: the rank gate rides the same clock (identity: no wr
}
/// &lt;summary&gt;The fresh dean: an unlicensed clinic, one bed, three
/// patients already queuing at the street corner.&lt;/summary&gt;
public static HospitalStage NewDean()
    =&gt; new HospitalStage(new HospitalIntake(1));
// ---- live faces ----
public double SimTime =&gt; _t;
public HospitalPhase Phase
{
    get
    {
        if (_laughDone) return HospitalPhase.FirstLaughter;
        if (_firstCoin) return HospitalPhase.FirstCoin;
        if (_deptOpen) return HospitalPhase.ClinicOpen;
        return HospitalPhase.Hook;
    }
}
public HospitalIntake Intake =&gt; _intake;
public HospitalEvents Events =&gt; _events;
public HospitalRank Rank =&gt; _rank;
public bool ModalActive =&gt; false;              // zero login wall, zero blocking modal (red line 11)
public bool CanDecide =&gt; _deptOpen == false;  // the doors decision is live right after the hook
public IReadOnlyList&lt;(string id, double t)&gt; Stamps =&gt; _stamps;
/// &lt;summary&gt;First stamp time of an event id (-1 when never fired).&lt;/summary&gt;
public double TimeOf(string id)
{
    for (int i = 0; i &lt; _stamps.Count; i++)
        if (_stamps[i].id == id) return _stamps[i].t;
    return -1;
}
void Stamp(string id)
```

```
{
    _stamps.Add((id, _t));
}
/// &lt;summary&gt;Advances the opening clock: drives the intake economy
/// and fires the scheduled beats (registration admit, first laugh,
/// review preview) on the frozen script. The dt is sliced at the
/// next scheduled beat so every promise lands on its scripted sim
/// time no matter how coarse the caller drives the clock (a whole
/// 13s hop still books the +0.5s admit on time - the bed turn then
/// completes before a review at t+13, the ladder walk rides this).&lt;/summary&gt;
public void Advance(double dt)
{
    if (dt &lt;= 0) return;
    if (TimeOf("evt_hospital_hook") &lt; 0)
        Stamp("evt_hospital_hook");   // the hook owns t0 (facade + street queue + caption)
    _admitBlocked = false;   // the registration retry law is per-call (a refused admit waits for the next Advan
    double remaining = dt;
    while (remaining &gt; 0)
    {
        double seg = remaining;
        double next = NextBeatTime();
        if (next - _t &lt; seg) seg = next - _t;   // slice at the beat, never past it
        if (seg &gt; 0)
        {
            _t += seg;
            _intake.Advance(seg);
            _events.Advance(seg);
            _rank.Advance(seg);   // T008: the reputation-band face tracks the live rep (own funnel, never write
            remaining -= seg;
        }
        FireScheduledBeats();
        if (seg &lt;= 0) remaining = 0;   // a due beat just resolved: hand control back to the caller, never sp
    }
}
/// &lt;summary&gt;The earliest pending scheduled beat (double.MaxValue
/// when the script has nothing left to fire). The blocked-admit
/// face keeps a refused registration out of the slice plan until
/// the next Advance call retries it (the old per-call law).&lt;/summary&gt;
double NextBeatTime()
{
    double next = double.MaxValue;
    if (_deptOpen &amp;&amp; !_firstCoin &amp;&amp; !_admitBlocked)
    {
        double admit = _deptOpenedAt + HospitalIntake.FirstAdmitDelaySeconds;
        if (admit &lt; next) next = admit;
    }
    if (_deptOpen &amp;&amp; !_laughDone &amp;&amp; FirstLaughAt &lt; next) next = FirstLaughAt;
    if (_deptOpen &amp;&amp; !_previewDone &amp;&amp; ReviewPreviewAt &lt; next) next = ReviewPreviewAt;
    return next;
}
```

```
/// &lt;summary&gt;Fires every scheduled beat whose scripted time the
/// clock has reached.&lt;/summary&gt;
void FireScheduledBeats()
{
    // the queue head steps in half a second after the doors open
    if (_deptOpen &amp;&amp; !_firstCoin &amp;&amp; !_admitBlocked
        &amp;&amp; _t &gt;= _deptOpenedAt + HospitalIntake.FirstAdmitDelaySeconds)
    {
        _firstCoinGoldBefore = _intake.GoldHc;
        if (_intake.TryRegisterQueueHead())
        {
            _firstCoinJumpHc = _intake.GoldHc - _firstCoinGoldBefore;   // snapshot at the stamp tick
            Stamp("evt_first_registration");
            Stamp("coin_visual");   // the coin popup face
            Stamp("coin_sfx");      // the signature coin chime trigger
            Stamp("coin_number");   // the gold counter jump
            Stamp("evt_first_coin");
            _firstCoin = true;
            _firstPatientDiag = HospitalTexts.EnText(HospitalTexts.KeyDiagHiccupBug);
        }
        else _admitBlocked = true;   // queue or bed refused: retry on the next Advance call
    }
    // the first laugh: the first fictional patient portrait + the absurd diagnosis line
    if (_deptOpen &amp;&amp; !_laughDone &amp;&amp; _t &gt;= FirstLaughAt)
    {
        Stamp("evt_first_laugh");
        Stamp("evt_first_patient_portrait");
        Stamp("evt_first_diag_line");
        _laughDone = true;
    }
    // the first review preview card (the level-1 review teaser)
    if (_deptOpen &amp;&amp; !_previewDone &amp;&amp; _t &gt;= ReviewPreviewAt)
    {
        Stamp("evt_review_preview");
        _previewDone = true;
    }
}
/// &lt;summary&gt;The doors decision: opens the first preset department
/// slot (pick WHAT opens). Returns false while the wallet or the
/// rank chain refuses.&lt;/summary&gt;
public bool OpenFirstDepartment(int choice)
{
    if (_deptOpen) return false;
    if (TimeOf("evt_hospital_hook") &lt; 0) return false;   // the hook owns t0
    if (!_intake.OpenNextSlot(choice)) return false;
    _deptOpen = true;
    _deptOpenedAt = _t;
    Stamp("evt_dept_opened");
    Stamp("evt_first_decision");
    return true;
```

```
        }
        public int FirstCoinJumpHc =&gt; _firstCoin ? _firstCoinJumpHc : 0;
        public string FirstPatientDiag =&gt; _firstPatientDiag;
        /// &lt;summary&gt;Requests the rank review (T008 host face): the engine
        /// owns the verdict and both host folds (the economy row swap and
        /// the stage pool widen land inside the settle). The opening script
        /// never calls this - the first review is a player action, so the
        /// signature law holds.&lt;/summary&gt;
        public ReviewVerdict RequestReview() =&gt; _rank.RequestReview();
        /// &lt;summary&gt;Opening fingerprint: every stamp, the wallet and the
        /// intake state - byte-identical across identical scripts.&lt;/summary&gt;
        public string OpeningSig()
        {
            var sb = new System.Text.StringBuilder();
            foreach (var s in _stamps) sb.Append(s.id).Append('@').Append(s.t.ToString("0.###")).Append(';');
            sb.Append(_intake.Fingerprint());
            return sb.ToString();
        }
    }
}
// HospitalTexts.cs
// HospitalTexts.cs - the hospital copy law for Crazy Hospital (G09-T004).
// English source strings live here as compiled code (the family source law:
// code strings stay English); the zh-CN localization rides a separate data
// file (Assets/_Game/Configs/Texts_zhCN.json - data, not source). Lookup
// follows the shared LocaleService contract: a missing key falls back to the
// key itself (machine-readable, never a crash), and a zh entry missing from
// the data file falls back to the English source. Zero scattered copy: every
// UI-facing sentence routes through these keys - DataSchemaCheck scans the
// runtime tree for stray CJK and proves the coverage law. The medical
// disclaimer is a permanent resident key (compliance face, always on).
using System;
using System.Collections.Generic;
using Sy.Core;
namespace Sy.CrazyHospital
{
    public static class HospitalTexts
    {
        // ---- key constants (the compiled copy surface) ----
        public const string KeyHookCaption = "hook_caption";
        public const string KeyMedicalDisclaimer = "medical_disclaimer";   // the permanent compliance face
        public const string KeyRank1 = "rank_1";
        public const string KeyRank2 = "rank_2";
        public const string KeyRank3 = "rank_3";
        public const string KeyRank4 = "rank_4";
        public const string KeyRank5 = "rank_5";
        public const string KeyStatPatients = "stat_patients";
        public const string KeyStatReputation = "stat_reputation";
        public const string KeyStatCureRate = "stat_cure_rate";
        public const string KeyStatGold = "stat_gold";
```

```
    public const string KeyStatDepts = "stat_depts";
    public const string KeyDeptOpen = "dept_open";
    public const string KeyDeptSlots = "dept_slots";
    public const string KeyRegFee = "reg_fee";
    public const string KeyBedFee = "bed_fee";
    public const string KeyRepBad = "rep_bad";
    public const string KeyRepPlain = "rep_plain";
    public const string KeyRepGood = "rep_good";
    public const string KeyRepBanner = "rep_banner";
    public const string KeyReviewLevel1 = "review_level1";
    public const string KeyReviewLevel2 = "review_level2";
    public const string KeyReviewLevel3 = "review_level3";
    public const string KeyReviewTop = "review_top";
    public const string KeyGatePromote = "gate_promote";
    public const string KeyGateRectify = "gate_rectify";
    public const string KeyResearchTitle = "research_title";
    public const string KeyDrugHiccupCola = "drug_hiccup_cola";
    public const string KeyDrugSleepwalkPatch = "drug_sleepwalk_patch";
    public const string KeyDrugShockSugar = "drug_shock_sugar";
    public const string KeyDisHiccupBug = "disease_hiccup_bug";
    public const string KeyDisGummyBone = "disease_gummy_bone";
    public const string KeyDisMonday = "disease_monday";
    public const string KeyDisHeadGrass = "disease_head_grass";
    public const string KeyDisCactusAccident = "disease_cactus_accident";
    public const string KeyDiagHiccupBug = "diag_hiccup_bug";
    public const string KeyDiagGummyBone = "diag_gummy_bone";
    public const string KeyDiagMonday = "diag_monday";
    public const string KeyDiagHeadGrass = "diag_head_grass";
    public const string KeyDiagCactusAccident = "diag_cactus_accident";
    public const string KeyAlbumTitle = "album_title";
    public const string KeyOfflineSummary = "offline_summary";
    public const string KeyOfflineDouble = "offline_double";
    public const string KeyNightTide = "night_tide";
    public const string KeyCarryLabel = "carry_label";
    public const string KeyNewJourney = "new_journey";
    public const string KeyNewJourneyConfirm = "new_journey_confirm";
    public const string KeyShareChallenge = "share_challenge";
    public const string KeyDailyReport = "daily_report";
    // ---- the quack-doctor hiring face (T006, SYS03) ----
    public const string KeyDoctorDraw = "doctor_draw";
    public const string KeyDoctorOnStaff = "doctor_on_staff";
    public const string KeyDutyRoster = "duty_roster";
    public const string KeyDocThreesec = "doc_threesec";
    public const string KeyDocRemedyQuack = "doc_remedy_quack";
    public const string KeyDocMoyu = "doc_moyu";
    public const string KeyDocWangdali = "doc_wangdali";
    public const string KeyDocNightNurse = "doc_night_nurse";
    public const string KeyDocBeautyChief = "doc_beauty_chief";
    public const string KeyQuirkThreesec = "quirk_threesec";
    public const string KeyQuirkRemedyQuack = "quirk_remedy_quack";
```

```
public const string KeyQuirkMoyu = "quirk_moyu";
public const string KeyQuirkWangdali = "quirk_wangdali";
public const string KeyQuirkNightNurse = "quirk_night_nurse";
public const string KeyQuirkBeautyChief = "quirk_beauty_chief";
// ---- the event-engine face (T007, SYS04): card titles, the
// inspection preview and the two choice labels ----
public const string KeyEventPreview = "event_preview";
public const string KeyEventChoiceGo = "event_choice_go";
public const string KeyEventChoiceSlack = "event_choice_slack";
public const string KeyEventE01 = "event_e01";
public const string KeyEventE02 = "event_e02";
public const string KeyEventE03 = "event_e03";
public const string KeyEventE04 = "event_e04";
public const string KeyEventE05 = "event_e05";
public const string KeyEventE06 = "event_e06";
public const string KeyEventE07 = "event_e07";
public const string KeyEventE08 = "event_e08";
public const string KeyEventE09 = "event_e09";
public const string KeyEventE10 = "event_e10";
// ---- the rank-review face (T008, SYS05/SYS06): the promotion
// ceremony show and the rectification comedy show beats ----
public const string KeyCeremonyShow = "ceremony_show";
public const string KeyRectifyShow = "rectify_show";
/// &lt;summary&gt;English source table (compiled - the source-of-law face).&lt;/summary&gt;
public static readonly Dictionary&lt;string, string&gt; En = new Dictionary&lt;string, string&gt;
{
    { KeyHookCaption, "WELCOME TO CRAZY HOSPITAL - YOUR DEAN LIFE STARTS NOW" },
    { KeyMedicalDisclaimer, "FOR ENTERTAINMENT ONLY - NOT MEDICAL ADVICE" },
    { KeyRank1, "UNLICENSED CLINIC" },
    { KeyRank2, "LEVEL-1 HOSPITAL" },
    { KeyRank3, "LEVEL-2 HOSPITAL" },
    { KeyRank4, "LEVEL-3 HOSPITAL" },
    { KeyRank5, "TOP-GRADE STAMPED" },
    { KeyStatPatients, "PATIENTS" },
    { KeyStatReputation, "REPUTATION" },
    { KeyStatCureRate, "CURE RATE" },
    { KeyStatGold, "GOLD" },
    { KeyStatDepts, "DEPTS" },
    { KeyDeptOpen, "OPEN A DEPARTMENT" },
    { KeyDeptSlots, "SLOTS: {0}/{1}" },
    { KeyRegFee, "REG FEE" },
    { KeyBedFee, "BED FEE" },
    { KeyRepBad, "REVIEW STORM - THE STREETS GO QUIET" },
    { KeyRepPlain, "QUIET STREETS, BUSINESS AS USUAL" },
    { KeyRepGood, "STEADY PATIENTS, STEADY LINES" },
    { KeyRepBanner, "BANNER-WALL QUEUES OUT THE DOOR" },
    { KeyReviewLevel1, "LEVEL-1 REVIEW (THE PAPERWORK LITERATURE PRIZE)" },
    { KeyReviewLevel2, "LEVEL-2 REVIEW (THE MYSTERY PATIENT STRIKES)" },
    { KeyReviewLevel3, "LEVEL-3 REVIEW (THE GREAT HYGIENE INSPECTION)" },
    { KeyReviewTop, "TOP-GRADE REVIEW (THE REHEARSED MIRACLE SHOW)" },
```

```
{ KeyGatePromote, "PROMOTED! {0} UNLOCKED" },
{ KeyGateRectify, "RECTIFY AND RESUBMIT - NEXT REVIEW WINDOW SOON" },
{ KeyResearchTitle, "WONDER-DRUG LAB" },
{ KeyDrugHiccupCola, "HICCUP COLA - THE BURP GETS LOUDER" },
{ KeyDrugSleepwalkPatch, "SLEEPWALK PATCH - WALK MORE, DREAM DEEPER" },
{ KeyDrugShockSugar, "SHOCK SUGAR - FULL-BODY PIXEL JITTER" },
{ KeyDisHiccupBug, "THE HICCUP BUG" },
{ KeyDisGummyBone, "GUMMY-BONE SYNDROME" },
{ KeyDisMonday, "MONDAY SYNDROME" },
{ KeyDisHeadGrass, "GRASS ON THE HEAD (CODER EDITION)" },
{ KeyDisCactusAccident, "CACTUS-DIET ACCIDENT" },
{ KeyDiagHiccupBug, "DIAGNOSIS: PRESCRIBE ONE MORE HICCUP COLA." },
{ KeyDiagGummyBone, "DIAGNOSIS: BONE IS NOW DESSERT. AVOID DESSERT." },
{ KeyDiagMonday, "DIAGNOSIS: SEVERE MONDAY. COME BACK ON FRIDAY." },
{ KeyDiagHeadGrass, "DIAGNOSIS: WATER TWICE A DAY, NO MOWING." },
{ KeyDiagCactusAccident, "DIAGNOSIS: THE CACTUS IS FINE. THE PATIENT IS NOT." },
{ KeyAlbumTitle, "FICTIONAL DISEASE ALBUM" },
{ KeyOfflineSummary, "WHILE YOU WERE GONE..." },
{ KeyOfflineDouble, "DOUBLE IT?" },
{ KeyNightTide, "NIGHT EMERGENCY TIDE!" },
{ KeyCarryLabel, "NEXT-DEAN CARRY: {0}" },
{ KeyNewJourney, "START A NEW DEAN LIFE" },
{ KeyNewJourneyConfirm, "TAP AGAIN - THE CARRY SEEDS YOUR NEXT CLINIC" },
{ KeyShareChallenge, "CAN YOUR HOSPITAL BEAT MINE?" },
{ KeyDailyReport, "DAY {0}: {1}" },
// ---- the quack-doctor hiring face (T006): names, mischief beats
// and the spree lines - pure comedy, zero medical claims ----
{ KeyDoctorDraw, "HIRE 10 DOCTORS (AD-FUNDED)" },
{ KeyDoctorOnStaff, "ALREADY ON PAYROLL - THE JOKE HIRES TWICE" },
{ KeyDutyRoster, "ON DUTY" },
{ KeyDocThreesec, "THREE-SECOND DOCTOR" },
{ KeyDocRemedyQuack, "FOLK-REMEDY QUACK" },
{ KeyDocMoyu, "SLACKING DIRECTOR" },
{ KeyDocWangdali, "WANG-DALI THE BONE BOLTER" },
{ KeyDocNightNurse, "NIGHT-SHIFT NURSE" },
{ KeyDocBeautyChief, "BEAUTY-INSTITUTE CHIEF" },
{ KeyQuirkThreesec, "NEXT! - THE THREE-SECOND CONSULT, QUEUE MOVES ITSELF" },
{ KeyQuirkRemedyQuack, "ANCESTRAL REMEDY, MYSTICAL EFFECTS - NO GUARANTEES, OBVIOUSLY" },
{ KeyQuirkMoyu, "THE VANISHING WARD ROUND - NOBODY SAW HIM ALL DAY" },
{ KeyQuirkWangdali, "BONES BOLTED ON LIKE SCREWS - DO NOT ASK" },
{ KeyQuirkNightNurse, "SLEEPWALKS THE FLAG PATROL AT 3 A.M. - NEVER TIRED, EVER" },
{ KeyQuirkBeautyChief, "WANTED A STAR FACE, GOT A PIXEL FACE" },
// ---- the event-engine face (T007): the comedy lives in the
// titles - five inspection cards + five daily cards ----
{ KeyEventPreview, "THE INSPECTORS ARE COMING - EVERYONE LOOK BUSY!" },
{ KeyEventChoiceGo, "PLAY IT STRAIGHT" },
{ KeyEventChoiceSlack, "WING IT" },
{ KeyEventE01, "SURPRISE HYGIENE INSPECTION (EVERYONE PRETEND TO WORK)" },
{ KeyEventE02, "THE LEADERSHIP VISIT (THE REHEARSED MIRACLE SHOW)" },
{ KeyEventE03, "THE MYSTERY PATIENT (A SERVICE-ATTITUDE EXAM)" },
```

```
            { KeyEventE04, "REVIEW-PAPER LITERATURE PRIZE (NEGATIVE-GROWTH OPTIMIZATION)" },
            { KeyEventE05, "FIRE-DRILL STRETCHER RELAY (WANG-DALI CARRIES THREE)" },
            { KeyEventE06, "THE SLEEPWALKING HERO ROUNDS THE WARD AND HANGS A BANNER" },
            { KeyEventE07, "THE HICCUP KID STARTS A HOSPITAL-WIDE HICCUP TIDE" },
            { KeyEventE08, "THE CACTUS REGULAR (A SPINY ATTRIBUTION COMEDY)" },
            { KeyEventE09, "THE OVERTIMED CODER GROWS GRASS" },
            { KeyEventE10, "THE MIRROR PRINCESS ORDERS A STAR FACE (GETS A PIXEL FACE)" },
            // ---- the rank-review face (T008): the two show beats ----
            { KeyCeremonyShow, "THE STAMP CEREMONY! FIREWORKS OVER THE COURTYARD, A RAIN OF BANNERS" },
            { KeyRectifyShow, "QUACK-REMEDY SIDE EFFECT - THE REVIEW EXPERT CHECKS INTO OUR OWN WARD 3" },
        };
        /// &lt;summary&gt;English lookup (missing key falls back to the key itself
        /// - machine-readable, never a crash).&lt;/summary&gt;
        public static string EnText(string key)
            =&gt; En.TryGetValue(key, out var v) &amp;&amp; !string.IsNullOrEmpty(v) ? v : key;
        /// &lt;summary&gt;Parses the zh-CN data file payload into a key-to-text
        /// map (values must be non-empty strings; anything else drops the
        /// entry so the lookup degrades to English, never to garbage).&lt;/summary&gt;
        public static Dictionary&lt;string, string&gt; ParseZh(string json)
        {
            var map = new Dictionary&lt;string, string&gt;();
            if (string.IsNullOrEmpty(json)) return map;
            try
            {
                if (!(MiniJson.Deserialize(json) is Dictionary&lt;string, object&gt; d)) return map;
                foreach (var kv in d)
                    if (!string.IsNullOrEmpty(kv.Key) &amp;&amp; kv.Value is string s &amp;&amp; !string.IsNullOrEmpty(s
                        map[kv.Key] = s;
            }
            catch
            {
                // corrupt data file = empty map = English fallback face
                return new Dictionary&lt;string, string&gt;();
            }
            return map;
        }
        /// &lt;summary&gt;LocaleService-compatible lookup delegate over a parsed
        /// zh map: zh-CN hits read the data file, everything else (and any
        /// miss) reads the English source. Wire-ready for the T005 UI face.&lt;/summary&gt;
        public static Func&lt;string, string, string&gt; LookupWithZh(Dictionary&lt;string, string&gt; zh)
        {
            return (lang, key) =&gt;
            {
                if (lang == "zh-CN" &amp;&amp; zh != null &amp;&amp; zh.TryGetValue(key, out var v) &amp;&amp; !string.I
                    return v;
                return EnText(key);
            };
        }
    }
}
```